SAPIENZA
UNIVERSITÀ DI ROMA

Sapienza University of Rome

Dipartimento di ingegneria informatica, automatica e gestionale "Antonio Ruberti"
Corso di Master Degree in Artificial Intelligence and Robotics

Multi-agent Decision Support System for Space Mission Operations

Thesis Advisor
Prof. Danilo Comminiello

Candidate
Antonio Lisa Lattanzio
2154208

Academic Year MMXXV-MMXXVI

Abstract

Lorem ipsum dolor sit amet, consectetur adipisicing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

Keywords: lorem, ipsum, dolor, sit, amet, consectetur, adipisicing, elit, sed, do.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

Chapter 2

The Evolution of Natural Language Processing and Decision Support Systems

The field of Natural Language Processing (NLP) has experienced a remarkable evolution over the past decades. Early approaches relied on manually engineered representations and statistical models, where language was treated as a collection of symbolic features rather than as a structured and contextual system. Advances in machine learning progressively shifted the focus toward distributed representations and neural architectures capable of learning linguistic patterns directly from data. This progression ultimately culminated in the Transformer architecture, which forms the foundation of modern Large Language Models (LLMs). Understanding this historical evolution is essential for appreciating the capabilities and limitations of current AI-based Decision Support Systems (DSS).

The development of NLP can be broadly divided into three major phases. The first phase is characterized by sparse statistical representations such as Bag-of-Words and n-gram language models. The second phase introduced recurrent neural architectures capable of modeling sequential dependencies. Finally, the third phase replaced recurrence with attention mechanisms, enabling scalable contextual reasoning over long sequences.

2.1 Early Symbolic Approaches

Early NLP systems were primarily based on symbolic representations and sparse statistical features. Rather than learning semantic representations from data, documents were encoded through manually designed features derived from word occurrences. One of the most influential representations was the *Bag-of-Words* (BoW) model [40, 49]. In this representation, each document is transformed into a vector containing the frequency of every word in a predefined vocabulary, while completely disregarding grammar, syntax, and word order.

For example, consider the following two sentences:

"The doctor treats the patient."

"The patient treats the doctor."

Although the semantic meaning of the two sentences is clearly different, the Bag-of-Words representation assigns them nearly identical vectors because they contain exactly the same set of words. The only retained information is the frequency of each term, whereas all structural information is discarded. Consequently, language is reduced to an unordered multiset of independent lexical features.

This can be explicitly observed in their Bag-of-Words representations, as shown in Table 2.1.

Term	Sentence 1	Sentence 2
the	2	2
doctor	1	1
patient	1	1
treats	1	1

Table 2.1: Bag-of-Words representation of two syntactically different but lexically identical sentences.

Although the semantic meaning of the two sentences is clearly different, the Bag-of-Words representation assigns them nearly identical vectors because they contain exactly the same set of words. The only retained information is the frequency of each term, whereas all structural information is discarded. Consequently, language is reduced to an unordered multiset of independent lexical features.

Despite this limitation, Bag-of-Words proved remarkably effective for several tasks, including document classification, spam detection, topic categorization, and information retrieval. Its simplicity, low computational cost, and compatibility with classical machine learning algorithms such as Naïve Bayes and Support Vector Machines [40] contributed to its widespread adoption.

However, the independence assumption underlying BoW prevents the model from capturing compositional meaning or contextual relationships between words. Expressions such as "artificial intelligence", "credit card", or "New York" are decomposed into independent terms, ignoring the fact that their meaning emerges from the combination of multiple words rather than from each individual token.

The statistical foundations of these early methods were strongly influenced by the *distributional hypothesis*, originally proposed by Harris [26], which states that words occurring in similar linguistic contexts tend to exhibit similar semantic properties. This principle later became one of the cornerstones of statistical NLP and distributional semantics.

Motivated by the need to incorporate sequential information, researchers introduced *n-gram language models* [21]. Instead of treating words as independent observations, n-gram models estimate the probability of a word by conditioning only on the previous $n-1$ tokens according to the Markov assumption:

$$P(w_t \mid w_1, \ldots, w_{t-1}) \approx P(w_t \mid w_{t-(n-1)}, \ldots, w_{t-1})$$

Depending on the value of n, different language models can be obtained. A unigram model ($n = 1$) assumes complete independence between words and is therefore equivalent to the Bag-of-Words assumption. A bigram model predicts each word using only the immediately preceding token, whereas a trigram model considers the previous two words.

For instance, given the sentence

"The patient needs medical assistance."

the extracted n-grams are:

- Unigrams: {The, patient, needs, medical, assistance}
- Bigrams: {The patient, patient needs, needs medical, medical assistance}
- Trigrams: {The patient needs, patient needs medical, needs medical assistance}

Unlike Bag-of-Words, these representations preserve a limited notion of local word order, allowing statistical models to learn common expressions, collocations, and short syntactic structures.

Nevertheless, n-gram models remain fundamentally constrained by their fixed context window. Long-range dependencies cannot be modeled because only the previous $n-1$ tokens are considered. For example, in the sentence

"The physician who examined the patient yesterday **prescribes** the medication."

the relationship between the subject “physician” and the verb “prescribes” spans several intermediate words. A small bigram or trigram model cannot capture such dependencies effectively.

A second limitation concerns scalability. As the value of n increases, the number of possible word combinations grows exponentially. Consequently, even very large corpora contain only a small fraction of all possible n-grams, leading to severe data sparsity problems.

These shortcomings motivated the transition toward neural language models capable of learning continuous distributed representations instead of relying exclusively on explicit frequency counts.

2.2 Early Neural Approaches

To overcome the limitations of n-gram models, early neural approaches introduced the idea of learning distributed representations over sequences. Simple Recurrent Networks (SRNs) by Elman [17] established a foundational architecture for processing sequential data and are considered a precursor to modern Recurrent Neural Networks (RNNs).

The training of neural networks, including recurrent architectures, is based on backpropagation [52]. In the recurrent setting, this is extended to Backpropagation Through Time (BPTT) [24], which enables gradient-based optimization over temporal dependencies by unrolling the network across time.

Unlike feedforward models, RNNs process input tokens sequentially while maintaining a hidden state that acts as a memory of past information. A standard formulation of an RNN is given by:

$$h_t = f(W_h h_{t-1} + W_x x_t + b_h)$$
$$y_t = W_y h_t + b_y$$

This formulation allows, in principle, the modeling of arbitrarily long dependencies through the recurrent hidden state. However, training RNNs in practice is challenging due to the vanishing and exploding gradient problems, which limit their ability to preserve information over long sequences.

To understand this issue, consider a simplified recurrent formulation:

$$h_t = \phi(W h_{t-1})$$

Under BPTT, the gradient of the loss with respect to a hidden state can be expressed as a product of Jacobians across time:

$$\frac{\partial L}{\partial h_t} = \frac{\partial L}{\partial h_T} \prod_{k=t}^{T-1} \frac{\partial h_{k+1}}{\partial h_k}$$

Taking the norm yields:

$$\left\| \frac{\partial L}{\partial h_t} \right\| \approx \prod_{k=t}^{T-1} \left\| \frac{\partial h_{k+1}}{\partial h_k} \right\|$$

This shows that the gradient magnitude is determined by repeated multiplication of Jacobian matrices across time steps. Depending on their spectral properties, this product can either grow or decay exponentially, leading respectively to exploding or vanishing gradients.

This fundamental limitation significantly hinders the ability of vanilla RNNs to learn long-term dependencies, and motivates the development of more advanced recurrent architectures designed to improve gradient flow over long sequences.

To address these issues, more advanced architectures were proposed, most notably Long Short-Term Memory (LSTM) networks [28]. LSTMs introduce an explicit memory cell c_t designed to preserve long-term information, together with a set of gating mechanisms that regulate information flow. The core idea is to decouple long-term memory from short-term hidden representations.

The LSTM update equations are defined as:

$$\begin{aligned}
I_t &= \sigma(X_t W_{xi} + H_{t-1} W_{hi} + b_i) \\
F_t &= \sigma(X_t W_{xf} + H_{t-1} W_{hf} + b_f) \\
O_t &= \sigma(X_t W_{xo} + H_{t-1} W_{ho} + b_o) \\
\tilde{C}_t &= \tanh(X_t W_{xc} + H_{t-1} W_{hc} + b_c) \\
C_t &= F_t \odot C_{t-1} + I_t \odot \tilde{C}_t \\
H_t &= O_t \odot \tanh(C_t)
\end{aligned}$$

The key property of this formulation is the additive update of the cell state c_t, which contrasts with the purely multiplicative transformations of standard RNNs. This structure allows gradients to flow more effectively through time, since the derivative of c_t with respect to c_{t-1} is modulated by the forget gate f_t, which can learn to preserve information when needed.

As a result, LSTMs significantly mitigate the vanishing gradient problem and enable learning of long-range dependencies that are difficult to capture with vanilla RNNs. This makes them particularly effective in sequence modeling tasks such as machine translation, speech recognition, and sequence labeling.

A further simplification of this architecture led to Gated Recurrent Units (GRUs), which combine the forget and input gates into a single update mechanism. GRUs maintain similar performance to

LSTMs in many tasks while reducing computational complexity and the number of parameters.

The GRU formulation is defined as:

$$
\begin{aligned}
Z_t &= \sigma(X_t W_{xz} + H_{t-1} W_{hz} + b_z) \\
R_t &= \sigma(X_t W_{xr} + H_{t-1} W_{hr} + b_r) \\
\tilde{H}_t &= \tanh(X_t W_{xh} + (R_t \odot H_{t-1}) W_{hh} + b_h) \\
H_t &= (1 - Z_t) \odot H_{t-1} + Z_t \odot \tilde{H}_t
\end{aligned}
$$

Both architectures represent a critical improvement over vanilla RNNs by explicitly addressing the instability of long-term temporal dependencies.

Despite their success, RNN-based architectures still process sequences sequentially, limiting parallelization during training and inference. This inherent inefficiency, combined with residual difficulties in modeling extremely long dependencies, eventually motivated the development of attention-based models and Transformers, which replaced recurrence with global context modeling.

This progression from BoW and n-gram models to RNNs and gated variants marks a fundamental shift in NLP: from static, order-agnostic representations toward dynamic, stateful systems capable of modeling language as a temporal process. This evolution forms the conceptual bridge toward modern contextual embedding methods and Transformer-based architectures.

2.3 From Static Embeddings to Sequential Modeling

While recurrent architectures addressed the problem of modeling sequential dependencies, they still relied on input representations that did not explicitly capture semantic similarity between words. This motivated the development of distributed word representations, commonly referred to as word embeddings. Building upon the distributional hypothesis, neural language models sought to learn continuous vector representations in which semantically related words occupy nearby regions of the embedding space. This led to the first fundamental advancement in how machines represent language: the introduction of *Word Embeddings*.

Instead of treating words as isolated discrete entities (as in BoW or classical n-grams), Bengio et al. [4] proposed mapping each word i in a vocabulary V to a continuous, real-valued vector $C(i) \in \mathbb{R}^m$ (where $m \ll |V|$), stored in a shared embedding matrix C. A feedforward neural network then processes the concatenation of these vectors within a fixed context window to predict the probability of the next token w_t via a *softmax* layer. The architecture allows semantically similar words are forced to settle in nearby regions of this continuous vector space, allowing the model to automatically generalize from a training sentence (e.g., *"the cat is walking"*) to unseen variations (e.g., *"a dog was running"*).

Although theoretically revolutionary, this early model suffered from a massive computational bottleneck caused by computing the softmax normalization over the entire vocabulary $|V|$ at each step. Nevertheless, it laid the conceptual foundations for modern representation learning.

The most influential work in this area is **Word2Vec**, proposed by Mikolov et al. [42]. This model demonstrated that words could be mapped into a high-dimensional vector space where semantic relationships are preserved as geometric distances.

The framework produces these embeddings through two distinct model architectures that leverage local context windows. The Continuous Bag-of-Words (CBOW) model learns representations by predicting a central target word w_t given its surrounding context words within a sliding window of maximum distance C. The vectors of the context words are projected and averaged into a single position, discarding structural word order. The training objective of CBOW is to maximize the average log probability:

$$\mathcal{L}_{\text{CBOW}} = \frac{1}{T}\sum_{t=1}^{T} \log P(w_t \mid w_{t-C}, \ldots, w_{t-1}, w_{t+1}, \ldots, w_{t+C}) \tag{2.1}$$

Conversely, the Continuous Skip-gram model reverses this logic by using the current central word w_t as input to predict the continuous vector representations of its surrounding context words. The Skip-gram objective function maximizes the following conditional log probability:

$$\mathcal{L}_{\text{Skip-gram}} = \frac{1}{T}\sum_{t=1}^{T} \sum_{-C \le j \le C, j \neq 0} \log P(w_{t+j} \mid w_t) \tag{2.2}$$

The empirical results presented in the study demonstrated significant breakthroughs in both scalability and semantic compositionality. Crucially, the resulting vector space demonstrated linear regularities, meaning that relationships between words were preserved as stable geometric translations. This allowed the models to solve complex semantic and syntactic analogies using simple algebraic operations, such as computing the vector $X = v_{\text{King}} - v_{\text{Man}} + v_{\text{Woman}}$ and finding that its closest cosine distance neighbor was exactly v_{Queen}. Evaluated across extensive benchmark datasets, CBOW achieved state-of-the-art performance on syntactic tasks such as tense and plural inflections, while Skip-gram significantly outperformed all previous architectures on complex semantic questions.

While Word2Vec enabled early NLP tasks to capture basic conceptual analogies, these embeddings remained **static**: a single vector was assigned to a word regardless of its context (e.g., "bank" in a financial vs. geographical sense). Moreover, Word2Vec does not model word order or syntactic structure, since embeddings are learned independently of the surrounding sentence. As a consequence, it cannot capture long-range dependencies or contextual nuances that are essential in many decision-support scenarios. The representation is therefore powerful at the lexical level but fundamentally limited for tasks requiring a deeper understanding of sequence dynamics.

Shortly after, Pennington et al. [47] introduced **GloVe** (Global Vectors for Word Representation), which addresses the structural limitations of local window-based methods by leveraging the global statistical information of the corpus. While Word2Vec relies on local context windows, GloVe explicitly constructs a word-word co-occurrence matrix X, where entries X_{ij} record how frequently word j appears in the context of word i.The core insight of GloVe is that the ratio of co-occurrence probabilities P_{ik}/P_{jk}—rather than the probabilities themselves—carries more meaningful semantic information. To capture this in a linear vector space, the authors propose a weighted least-squares model that seeks to satisfy the following objective:

$$J = \sum_{i,j=1}^{V} f(X_{ij}) \left(w_i^T \tilde{w} j + b_i + \tilde{b} j - \log Xij \right)^2 \tag{2.3}$$

In this formulation, w_i and $\tilde{w}_j$ represent the target and context word vectors, while b_i and $\tilde{b}_j$ are bias terms that account for individual word frequencies. The function $f(Xij)$ acts as a weighting mechanism, ensuring that rare co-occurrences are not overweighted, while preventing common word pairs (e.g., "the", "and") from dominating the training objective.By training directly on the non-zero elements of the co-occurrence matrix, GloVe synthesizes the efficiency of global matrix factorization with the superior semantic capture of local window models. Empirical evidence confirmed this hybridization: GloVe demonstrated significant improvements in word analogy tasks and named entity recognition, consistently outperforming its predecessors by producing a vector space with a more robust and meaningful substructure. However, like its predecessors, GloVe remains a **static** embedding model, providing a single vector representation for each word regardless of the polysemy inherent in natural language.

While traditional word embedding models, such as Word2Vec, have proven successful in mapping semantic relationships into continuous vector spaces, they rely on treating words as atomic and isolated entities. This approach ignores the internal morphological structure of language, which represents a significant limitation for morphologically rich languages and results in the challenge of out-of-vocabulary terms, where rare words fail to acquire reliable representations. To overcome these constraints, the FastText framework [8] was introduced as a powerful extension of the skip-gram architecture that explicitly incorporates subword information. The core innovation of this approach lies in the decomposition of each word into a set of character n-grams. By adding special boundary symbols to distinguish prefixes and suffixes, the model represents every word as the sum of the vector representations of its constituent character n-grams. This design provides several critical advantages that distinguish FastText from previous static embedding methods. First, it ensures high robustness to rare words and previously unseen tokens, as the model can generate meaningful vector representations for any word by composing the vectors of its known n-grams. Furthermore, the model demonstrates a sophisticated morphological awareness, effectively capturing semantic commonalities between inflected forms or complex compound words, an ability that proves particularly advantageous for morphologically rich languages where it consistently outperforms standard word-level approaches.

To address the issue of polysemy and contextual nuance, the field moved toward **Deep Contextualized Word Representations**. A landmark in this transition was **ELMo** (Embeddings from Language Models), proposed by Peters et al. [48]. Unlike static embedding models, ELMo represents each word as a function of its surrounding context by exploiting a deep bidirectional Language Model (biLM). Rather than relying solely on word-level lookup tables, ELMo constructs input representations from character-level convolutions (CNNs), allowing the model to capture rich morphological information and generate meaningful embeddings even for out-of-vocabulary words.

The biLM consists of multiple stacked bidirectional LSTM layers trained with a self-supervised language modeling objective. The forward language model predicts the next token given the preceding context, while the backward language model predicts the previous token given the following context. By processing the sequence in both directions, the model captures both short- and long-range dependencies. Lower layers tend to encode syntactic and morphological information, whereas deeper layers capture increasingly abstract semantic features.

Instead of using only the final hidden layer, ELMo computes the representation of the k-th token as a task-specific weighted combination of all internal biLM representations:

$$\text{ELMo}_k^{\text{task}} = \gamma^{\text{task}} \sum_{j=0}^{L} s_j^{\text{task}} \, h_{k,j}^{LM}$$

where $h_{k,j}^{LM}$ denotes the representation of the k-th token produced by the j-th layer of the bidirectional language model, s_j^{task} are task-specific normalized weights learned during downstream training, and γ^{task} is a trainable scaling parameter.

This weighted combination allows downstream models to dynamically exploit both lower-level syntactic information and higher-level semantic knowledge depending on the target task. Consequently, ELMo replaces the idea of a single fixed embedding per word with contextual representations that vary according to the surrounding sentence, substantially improving performance on tasks such as question answering, sentiment analysis, and textual entailment.

Although ELMo represented a major breakthrough by introducing contextual embeddings, it still relied on recurrent architectures to model sequences. Consequently, contextual representations had to be computed sequentially, limiting parallelization and making long-range dependency modeling computationally expensive. These limitations ultimately motivated the development of the Transformer architecture, which replaced recurrence with self-attention.

2.4 The Transformer Revolution

The definitive breakthrough occurred with the publication of *"Attention Is All You Need"* [60]. Vaswani et al. proposed a novel architecture entirely based on the **Self-Attention mechanism**, dispensing with recurrence and convolutions. The core innovation lies in the model's ability to weigh the significance of different parts of the input data independently of their distance in the sequence. Mathematically, the Scaled Dot-Product Attention is defined as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{2.4}$$

where Q, K, and V represent the Query, Key, and Value matrices, and d_k is the dimension of the keys. The scaling factor $\sqrt{d_k}$ plays a crucial role in stabilizing the optimization process. Without this normalization term, the dot products between high-dimensional query and key vectors could grow excessively large, causing the softmax function to produce extremely peaked probability distributions and resulting in vanishing gradients during training. The scaling operation therefore contributes to more stable learning dynamics and improved convergence properties.

Conceptually, the attention mechanism can be interpreted as a weighted aggregation process. Each query vector determines how much importance should be assigned to all available key-value pairs, enabling the model to selectively focus on the most relevant contextual information for a given token. This selective processing capability constitutes one of the fundamental reasons behind the remarkable effectiveness of Transformer-based architectures.

This mechanism allows the model to build a global understanding of the input, enabling the emergence of "contextual intelligence" where word representations change dynamically based on the surrounding text. In the context of NLP, this marked the transition from local pattern matching toward richer contextual representations capable of modeling long-range semantic and syntactic dependencies. These representations later proved essential for the emergence of increasingly sophis-

ticated reasoning-like behaviors in large-scale language models. A key extension introduced in the original Transformer architecture is the multi-head self-attention mechanism. Instead of computing a single attention map, the model projects the input into multiple sets of query, key, and value spaces and applies the attention function in parallel across these sets.

Formally, multi-head attention is defined as:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(head_1, \ldots, head_h)W^O \tag{2.5}$$

where

$$head_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \tag{2.6}$$

and W_i^Q, W_i^K, W_i^V, and W^O are learnable projection matrices. This formulation enables each attention head to specialize in capturing different forms of dependencies within the data, thereby enriching the overall representation learned by the model.

The outputs from these attention heads are then concatenated and linearly transformed, allowing the model to jointly attend to diverse types of relationships in the input sequence, such as syntactic and semantic patterns simultaneously. By combining multiple attention heads, the network can capture richer representations than a single attention head would allow, without increasing the overall model depth.

Each Transformer block combines multi-head self-attention with a position-wise feed-forward network (FFN). Residual connections and layer normalization are applied around each sub-layer, facilitating stable optimization and enabling the successful training of very deep architectures.

Although self-attention enables every token to attend to all other tokens in the sequence, the mechanism itself is inherently permutation invariant and therefore provides no information about token order. To preserve the sequential nature of language, the original Transformer introduces **Positional Encodings**, which are added to the input embeddings before they are processed by the encoder or decoder.

Rather than learning positional representations, Vaswani et al. proposed deterministic sinusoidal functions capable of encoding both absolute and relative positions within a sequence:

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right) \tag{2.7}$$

$$PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right) \tag{2.8}$$

where pos denotes the token position and d_{model} is the embedding dimension. This formulation allows the model to infer relative distances between tokens while also enabling generalization to sequence lengths not encountered during training. Although modern large language models often replace sinusoidal encodings with learned positional embeddings or rotary positional embeddings (RoPE), the original formulation remains a foundational contribution that demonstrated how sequential information could be incorporated without relying on recurrent architectures.

Beyond the attention mechanism itself, the original Transformer introduced a modular encoder-decoder architecture, illustrated in Figure 2.1. The encoder consists of a stack of identical layers, each composed of a multi-head self-attention module followed by a position-wise feed-forward net-

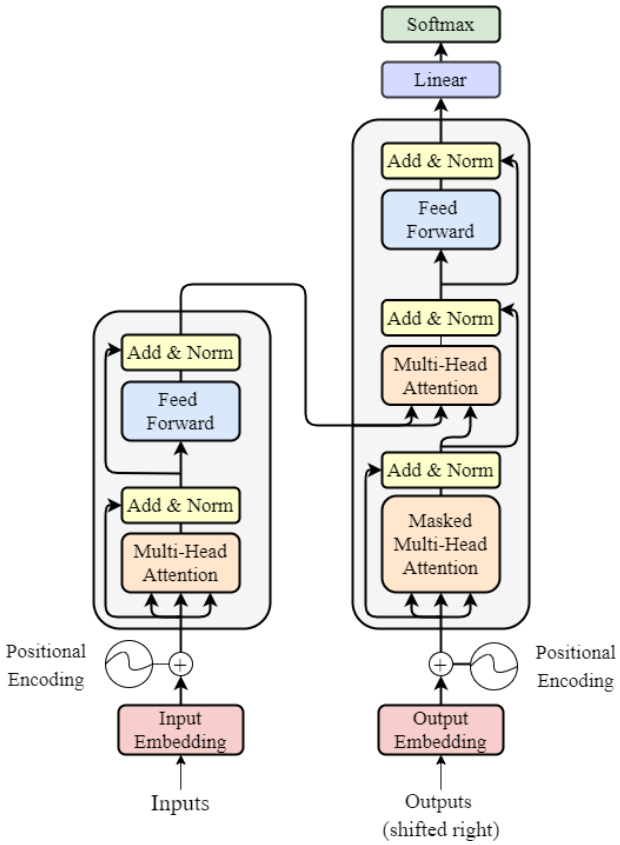

Figure 2.1: The original Transformer encoder-decoder architecture proposed by Vaswani et al. [60]. The figure illustrates the composition of encoder and decoder stacks, the multi-head attention modules, feed-forward layers, residual connections, and positional encodings. Source: reproduced from [60].

work. Residual connections and layer normalization surround each sub-layer, improving optimization stability and enabling deeper architectures.

The decoder follows a similar structure but incorporates two important differences. First, it employs masked self-attention to prevent each output token from attending to future positions during training. Second, an additional cross-attention layer allows the decoder to attend to the contextual representations generated by the encoder, enabling effective sequence-to-sequence learning for tasks such as machine translation.

This modular design later gave rise to the dominant language model families. Encoder-only architectures, such as BERT, specialize in language understanding tasks, while decoder-only architectures, such as the GPT series, are optimized for autoregressive text generation. Encoder-decoder models, including T5, remain particularly effective for sequence transformation tasks such as translation and summarization.

Another important practical advantage of Transformer architectures is their massive parallelization potential. Unlike recurrent networks, which process tokens sequentially, each attention layer in a Transformer can compute interactions between all positions in the input sequence simultaneously. As a result, Transformers can be trained more efficiently on large corpora, enabling the scale-up that underpins modern large language models.

Despite its advantages, the standard self-attention mechanism exhibits quadratic time and memory complexity with respect to the input sequence length, as every token attends to every other token. This limitation has motivated extensive research into efficient attention mechanisms like

flash attention [14], which uses tiling to reduce the number of memory reads/writes, or linformer [62], that approximate the standard attention mechanism with low rank matrices.

Collectively, these innovations established the Transformer as the dominant neural architecture for NLP and, subsequently, for a wide range of machine learning applications. More importantly, its scalability and training efficiency made it possible to train models on unprecedented amounts of data and parameters, laying the foundation for the emergence of modern Large Language Models.

2.5 Scaling Laws and Emergent Abilities

The evolution culminated in the transition from task-specific models to *Large Language Models* (LLMs). This shift was driven by the discovery of scaling laws [34], which suggested that model performance, and consequently capabilities on NLP tasks, improve with increases in parameter count, dataset size, and compute, which can indirectly improve the performance of systems that leverage NLP components, including DSS applications.

Furthermore, the introduction of the Pre-training and Fine-tuning paradigm, popularized by BERT [15] and the GPT series [50], enabled these systems to internalize vast amounts of world knowledge. This trajectory led to the discovery of *emergent abilities*: capabilities not explicitly present in smaller models, such as in-context learning and multi-step logical reasoning [64]. These behaviors are referred to as emergent because they are not explicitly programmed nor consistently observed in smaller models, but rather appear once the model exceeds certain scales of parameters and training data. Although the precise mechanisms underlying emergence remain an active area of research, empirical evidence suggests that increasing scale enables qualitatively new capabilities beyond simple improvements in benchmark accuracy. For modern Decision Support Systems, this represents a transition from "engines that process text" to systems that can generate knowledge-informed recommendations, while human oversight remains necessary.

2.6 Foundation Models and General-Purpose AI

The scalability of the Transformer architecture, coupled with the empirical discovery of scaling laws, has catalyzed a paradigm shift in artificial intelligence: the emergence of **Foundation Models**. As defined by Bommasani et al. [9], a foundation model is any model trained on a vast amount of data, typically using self-supervision at scale, that can be adapted (e.g., fine-tuned) to a wide range of downstream tasks. This represents a fundamental departure from the traditional AI development lifecycle, where models were painstakingly engineered for single, narrow objectives. Instead, foundation models serve as a versatile base, encoding broad representations of language, logic, and world knowledge that can be repurposed across diverse domains, including the sophisticated requirements of Decision Support Systems.

A pivotal demonstration of this potential was provided by Brown et al. [10] with the introduction of GPT-3. This work highlighted that when language models are scaled to billions of parameters, they begin to exhibit *in-context learning* capabilities. Unlike previous approaches that required gradient-based fine-tuning to perform a new task, GPT-3 demonstrated the ability to adapt to specific instructions simply by observing a few examples provided within the input prompt (few-shot learning). This shift fundamentally changed the interaction between users and systems, moving

from "training" to "prompting" as the primary mechanism for directing model behavior.

However, the rapid expansion of model size brought to light significant challenges regarding computational efficiency. The industry trend initially focused on increasing parameter counts indiscriminately; this was challenged by Hoffmann et al. [29] in their study on *Chinchilla*. The authors demonstrated that the performance of a language model is not solely dependent on its parameter count, but rather on an optimal balance between model size and the volume of training data. Their analysis showed that, for a fixed computational budget, allocating additional resources to training data rather than solely increasing parameter count yielded substantially better performance. This finding established new scaling laws, suggesting that most contemporary models were significantly under-trained. This insight provided a roadmap for developing "compute-optimal" models, enabling superior performance with more efficient resource allocation.

2.7 Alignment and Instruction Tuning

Although large language models acquired broad linguistic knowledge through large-scale pre-training, they were not inherently optimized to follow user instructions or assist with practical tasks like Decision Support Systems, this powerful capability was enabled by the development of *Alignment* techniques. While base LLMs possess extensive statistical information derived from their training corpora, they often fail to follow specific user intents or produce helpful, safe, and concise reasoning. To bridge this gap, **Instruction Tuning** was introduced, most notably through the work on **InstructGPT** by Ouyang et al. [44]. This process involves **fine-tuning** the model on a dataset of human-written prompt-response pairs, instruction tuning changes the training distribution by exposing the model to datasets composed of instructions paired with high-quality responses, thereby teaching it to better interpret and follow user requests.

A critical component of this alignment is **Reinforcement Learning from Human Feedback (RLHF)**. This methodology, popularized by Christiano et al. [12] and later refined for LLMs [58], allows models to optimize their outputs based on human preferences. The RLHF pipeline typically consists of three stages. First, a supervised fine-tuning phase is performed, where the initial language model is trained on high-quality demonstrations generated by human annotators. Second, a reward model is trained from human preference comparisons, where evaluators rank alternative model outputs according to criteria such as helpfulness, relevance, and overall quality. Finally, the language model is optimized against the learned reward model using reinforcement learning, typically through Proximal Policy Optimization (PPO) [55]. By incorporating human preferences into the optimization process, RLHF guides the model toward outputs that better satisfy alignment objectives, although limitations such as hallucinations, biases, and sensitivity to reward model imperfections remain. While RLHF has been the industry standard for alignment, its reliance on a separately trained reward model and the computational complexity of the Proximal Policy Optimization (PPO) algorithm have led to the development of more stable alternatives. A prominent breakthrough is **Direct Preference Optimization** (DPO), introduced by Rafailov et al. [51]. DPO simplifies the alignment process by mathematically deriving a direct mapping between the reward function and the optimal policy, effectively bypassing the need for a separate reward model and reinforcement learning training loops. By optimizing the language model directly on preference data via a simple classification loss, DPO offers a computationally efficient and more

stable approach to alignment, which has rapidly become a preferred method for fine-tuning modern Large Language Models.

2.8 The Reliability Gap: Hallucinations and Uncertainty

One of the primary obstacle to the integration of LLMs in critical Decision Support Systems (DSS) is the phenomenon of **hallucinations**, defined as the generation of factually incorrect or ungrounded statements despite high linguistic fluency [31]. This stems from the underlying training objective of next-token log-likelihood, which prioritizes plausible linguistic continuation over factual verification. In a DSS context, **extrinsic hallucinations**, where the output cannot be verified from provided sources or real-world knowledge, are particularly hazardous, as the model may provide a confident but false justification for a recommendation [3].

Beyond explicit errors, a critical risk is the **lack of self-awareness regarding uncertainty**. State-of-the-art models are often poorly calibrated, meaning their internal probability estimates do not align with their actual correctness [32].

Recent surveys [1] identify a broad taxonomy of **hallucination detection methods**, designed to assess the factuality and faithfulness of LLM outputs before they are used in high-stakes Decision Support Systems.

- **Retrieval-based methods** detect hallucinations by comparing generated statements against external knowledge sources or retrieved evidence. These techniques, often implemented through RAG pipelines [35], are particularly effective for identifying *extrinsic hallucinations* by checking whether claims are supported by authoritative documents.
- **Uncertainty-based approaches** estimate unreliability through internal model signals such as token-level probabilities, entropy, attention stability, or structured uncertainty propagation. This line of work directly addresses the calibration issues highlighted in [32], offering a principled way to detect when a model is uncertain despite fluent output.
- **Embedding-based methods** measure semantic alignment between inputs, outputs, and reference sources within a shared embedding space. Low semantic similarity indicates drift from the provided context or task grounding.
- **Learning-based detectors**, including supervised, weakly supervised, and agent-based systems, train dedicated models to classify hallucinated versus factual outputs. These techniques capture complex error patterns that cannot be identified through simple probability thresholds or similarity metrics.
- **Self-consistency-based techniques** generate multiple answers or paraphrased queries to assess stability across model outputs. Inconsistencies in meaning or reasoning serve as indicators of potential hallucinations, making these methods valuable when external verification is unavailable.

Overall, this taxonomy underscores that mitigating hallucinations involves not only filtering incorrect content but also understanding where and why LLM reasoning becomes unreliable, which is essential for safe integration into critical DSS.

Recent work suggests that some failure modes traditionally framed as hallucinations may come not only from generative uncertainty but also from structural challenges in long-context evidence aggregation [6]. Studies on retrieval in extended contexts show that the size and distribution of relevant information influence an LLM's ability to locate and integrate the correct evidence. When small but crucial "gold" contexts are embedded within much longer distractor passages, models exhibit reduced aggregation accuracy and increased positional sensitivity. These effects are particularly relevant for retrieval-augmented and agentic systems, where evidence sources often vary considerably in length. In such settings, short but important documents may be underutilized, potentially leading the model to produce unsupported outputs even when the necessary information is included in the input. These findings suggest that hallucination-related risks arise not only from calibration issues or insufficient grounding but also from context-size asymmetry and the design of evidence-aggregation pipelines.

2.9 Computational Burden and Compression Strategies for LLMs

Even though LLMs are extremely useful they also are computationally expensive, the development of new models has been charcterized by an increase of the parameters size that grown to hundreds of billions of parameters in the last years, with hardware that are getting more and more powerful and optimized to keep up with the evolution of this technology [36]. The main methods to reduce the cost are applicable during the fine-tuning stage or the inference task:

- Parameter-efficient fine-tuning (PEFT): refers to the process of adjusting the parameters of a pre-trained large model to adapt it to a specific task or domain while minimizing the number of additional parameters introduced or computational resources required [25]. One of their main methods is LoRA [30] which freezes the pre-trained model weights and injects trainable rank decomposition matrices into each layer of the Transformer architecture, reducing the number of trainable parameters.

- Parameters quantization: focuses on reducing the size of the parameters to usually 4 or 8 bits like Smoothquant [66], representative post-training quantization methods include [22], which quantizes weights block-wise and iteratively corrects quantization errors using Hessian information, and Activation-aware Weight Quantization (AWQ), which focuses on reducing the error by preserving the most important weights [38].

- Quantization Aware Fine Tuning: QA-Lora represents a hybrid approach between PEFT and quantization, where the model is fine-tuned while accounting for quantization constraints [67].

- Pruning: focus on lightening the model size by removing the connections or neurons that are less important [23, 57]

- Knowledge Distillation: is a technique where a smaller model (the student) is trained to replicate the behavior of a larger model (the teacher), effectively transferring knowledge from the teacher to the student [27, 53].

Compression techniques inevitably involve trade-offs between efficiency and performance, as aggressive quantization, pruning, or parameter-efficient fine-tuning may degrade model accuracy or

require additional implementation effort, and task-specific performance variability must be considered [22, 23, 30, 38].

2.10 Reasoning Frameworks for Decision Support

Beyond architectural and alignment improvements, the operational use of LLMs in decision-making has been revolutionized by specialized prompting and reasoning frameworks. The introduction of **Chain-of-Thought (CoT)** prompting [65] demonstrated that forcing a model to generate intermediate reasoning steps significantly enhances performance on complex logical tasks.

While CoT provides a "slow thinking" approach essential for auditing a DSS, relying on a single greedy-decoded trajectory can lead to error propagation. To mitigate this, **Self-Consistency** [63] introduces a decoding strategy that samples multiple reasoning paths and selects the most frequent marginal result, leveraging the intuition that complex problems often admit multiple paths to a unique correct answer.

Further advancing non-linear reasoning, the **Tree of Thoughts (ToT)** framework [68] generalizes the CoT paradigm by representing intermediate reasoning steps as discrete "thoughts" organized in a tree structure. Unlike CoT, where the model commits to a single sequential chain of deductions, ToT enables the exploration of multiple candidate reasoning trajectories through a search process. At each step, the LLM can generate alternative intermediate states, evaluate their quality through self-assessment or external feedback mechanisms, and prune unpromising branches. This approach introduces a form of deliberative reasoning closer to classical search algorithms, allowing the model to backtrack, compare alternative hypotheses, and allocate additional computational effort to particularly challenging decisions. In the context of Decision Support Systems (DSS), ToT is especially valuable for scenarios characterized by uncertainty, long planning horizons, or conflicting objectives, where premature commitment to a single reasoning path may lead to suboptimal recommendations.

Building upon this idea, the **Graph of Thoughts (GoT)** framework [5] relaxes the hierarchical constraints of tree-based exploration by modeling reasoning processes as directed graphs rather than strictly branching structures. While ToT assumes that each new thought derives from a single previous state, GoT allows arbitrary dependencies between intermediate reasoning units, enabling operations such as aggregation, merging, refinement, and transformation of partial solutions. This graph-based representation better captures complex problem-solving processes in which multiple reasoning paths can interact and contribute jointly to a final decision. For DSS applications, GoT provides a more flexible framework for representing interconnected evidence, alternative hypotheses, and multi-stage decision processes, where information may need to be combined, revisited, or propagated across different reasoning components.

The **ReAct** framework [69] represents a fundamental transition from isolated reasoning toward agentic decision-making by interleaving reasoning and action. Instead of generating a final answer based solely on the information encoded in the model parameters, ReAct structures the interaction as an iterative loop composed of three main phases: reasoning, action, and observation. The model first generates a reasoning trace to determine the next objective or strategy, then executes an external action, such as querying a database, retrieving documents, invoking an API, or interacting with a software environment, and finally incorporates the resulting observations into subsequent reasoning steps.

This mechanism allows LLMs to dynamically acquire information, verify intermediate assumptions, and adapt their behavior according to external feedback. Compared with purely reasoning-oriented approaches such as CoT or ToT, ReAct introduces an explicit connection between cognitive processes and environmental interaction, reducing the limitations caused by static knowledge and closed-world assumptions. For Decision Support Systems, this capability is particularly relevant because decisions often depend on continuously changing data sources, domain-specific tools, and feedback from operational environments.

When integrated with Retrieval-Augmented Generation (RAG) [35], ReAct-based systems can combine structured reasoning with external knowledge retrieval, enabling LLMs to identify information gaps, retrieve relevant evidence, evaluate available alternatives, and progressively refine recommendations. This combination shifts LLMs from static predictive models toward interactive reasoning agents capable of supporting complex decision processes in dynamic and uncertain environments.

2.11 Long-Context Modeling and Context Management

While early Transformer-based models were limited to relatively short context windows, modern LLMs have progressively expanded their capacity to process longer sequences. This evolution enables applications requiring the analysis of extensive documents, multi-turn conversations, and complex workflows involving heterogeneous information sources. However, the standard Transformer architecture suffers from a quadratic computational and memory complexity, $O(n^2)$, relative to sequence length n. This constraint turns the context window into a significant bottleneck: as sequence length increases, the memory footprint of the Key-Value (KV) cache grows linearly, often exhausting GPU VRAM during inference, while the self-attention mechanism becomes the primary computational latency driver. To overcome these limitations, recent research has focused on architecture and positional encoding advancements.

Initial approaches relied on linear attention approximations, but many state-of-the-art models primarily adopt **Rotary Positional Embeddings (RoPE)** [59]combined with scaling techniques. Works such as Position Interpolation [11] and **YaRN** [46] have demonstrated that context windows can be extended far beyond pre-training limits by mathematically re-scaling the rotational frequency of embeddings, allowing models to 'perceive' longer sequences without extensive retraining.

Another problem is the increased risk of hallucinations and reasoning errors when the model is forced to process long contexts, as the relevant information may be diluted among irrelevant content. This is particularly problematic in Decision Support Systems, where critical evidence may be embedded within lengthy documents or multi-source data streams. The work of Liu et al. [39] demonstrates that current language models, including those explicitly designed to handle long contexts, struggle to robustly retrieve and use information located in the middle of their input window. Instead of treating all parts of the context equally, these models exhibit a U-shaped performance curve where performance is highest when relevant information is placed at the very beginning (primacy bias) or at the very end (recency bias) of the input. Performance drops significantly when the model is forced to access information located in the middle of the context. In some cases, performance is even worse than if the model had been given no context at all.

2.12 Agentic Large Language Models

The evolution from Large Language Models to Agentic Large Language Models represents a shift from passive text generation toward autonomous task execution. While traditional LLMs operate primarily as reactive systems that map user prompts to generated responses, agentic architectures embed language models within broader computational systems that provide planning, memory, action executions, and, in some cases, profiling modules [61].

The **profiling module** defines the role, objectives, capabilities, and behavioral constraints of the agent. By specifying characteristics such as domain expertise, task objectives, or interaction style, the profile conditions the reasoning process and influences planning, memory retrieval, and action selection.

The **memory module** can store data in several formats, each offering unique trade-offs between semantic richness, retrieval efficiency, and ease of manipulation: Natural Language, Embeddings, Databases, Structured Lists, or hybrid. The selection of data is typically governed by three criteria: recency, relevance, and importance. Memory is commonly divided into short-term memory, which maintains the context of the current interaction, and long-term memory, which stores persistent knowledge, previous experiences, or retrieved information across multiple tasks [61]. Effective memory management is essential for long-horizon reasoning and continual adaptation. A notable implementation of this concept is provided by Generative Agents [45], where memories are organized as a continuously growing memory stream. Rather than retrieving past experiences indiscriminately, the agent ranks memories according to their recency, semantic relevance, and estimated importance. Furthermore, higher-level reflections are periodically synthesized from accumulated experiences, enabling the agent to form persistent knowledge that can guide future planning and decision-making.

The **planning module** is responsible for managing long-horizon tasks, orchestrating complex workflows, and ensuring coherent action sequences across multiple steps. The planning can be performed without feedback, where the agent does not get a feedback that influences its behaviour or with feedback for tasks where initial plans frequently fail due to unforeseen environmental dynamics or overly complex preconditions. The feedback can be provided by the environment, by other agents, or by a human-in-the-loop. Planning is commonly supported by reasoning strategies such as Chain-of-Thought (CoT), Tree of Thoughts (ToT), or Graph of Thoughts (GoT), which enable the agent to decompose complex objectives into intermediate reasoning steps before selecting executable actions.

The **action module** serves as the final downstream component of an autonomous agent architecture, translating high-level decisions into tangible outcomes through direct environmental interaction.

An agent's ability to execute these actions relies on a synergy between the LLM's inherent reasoning and a suite of external resources:

- Tool Integration: External tools bridge critical gaps in an LLM's capabilities, such as mitigating hallucinations or providing domain-specific expertise.
- APIs: Agents utilize APIs to interface with web services, calculators, compilers, and specialized software libraries, enabling them to perform precise, executable tasks.
- Databases and Knowledge Bases: allow agents to query structured data, ensuring that actions are grounded in verifiable, external information.

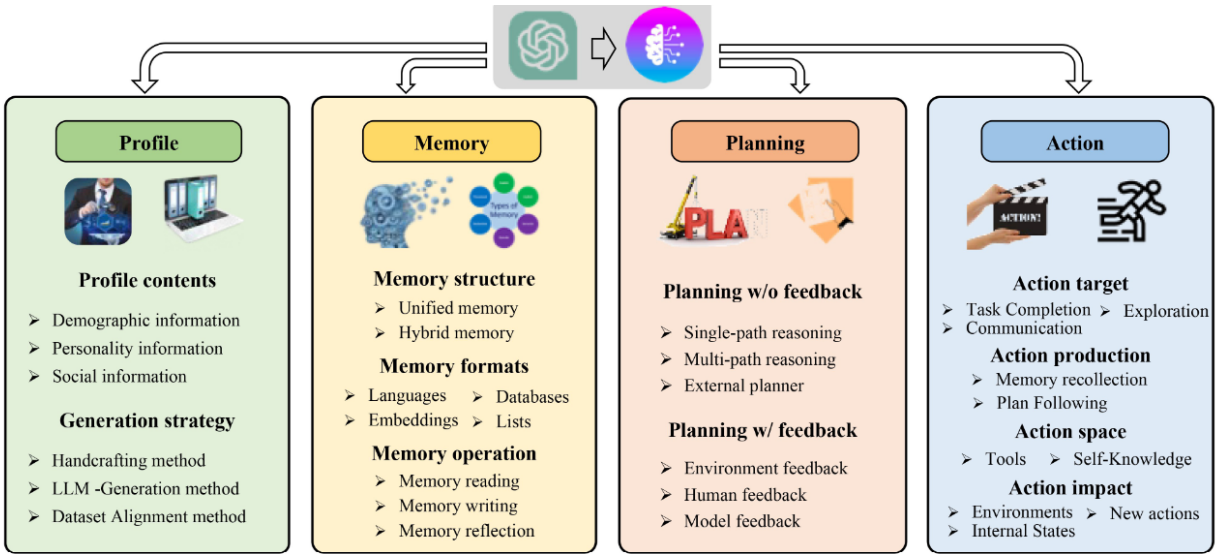

Figure 2.2: Unified framework proposed by [61] for LLM-based autonomous agents. Source: Wang et al.[61].

- External Models: Agents can offload complex or multimodal processing to specialized supplementary models.

Tool integration was further formalized by Toolformer [54], which demonstrated that language models can learn when and how to invoke external tools during generation through self-supervised training. This capability extends the model beyond its parametric knowledge by enabling access to calculators, search engines, translation systems, or APIs whenever additional information or computation is required. Despite using a relatively small base model (GPT-J with 6.7B parameters), Toolformer significantly outperformed larger models like GPT-3 (175B) on tasks requiring math or factual lookup because it knows when to "outsource" the work to a tool. Moreover, the model effectively chooses if and when to use a tool without being told to do so by the user.

One of the most influential frameworks for planning and interaction is ReAct (Reason + Act) [69], which interleaves reasoning steps with actions performed in the external environment. Rather than generating an entire solution in a single pass, the agent alternates between internal reasoning ("Thought"), executing an action (e.g., querying a search engine or database), and incorporating the resulting observations into subsequent reasoning. This iterative loop:

$$\text{Thought} \rightarrow \text{Action} \rightarrow \text{Observation}$$

enables adaptive decision-making and significantly improves performance on knowledge-intensive and interactive tasks.

Beyond planning and acting, recent agentic systems increasingly incorporate explicit self-reflection mechanisms. Reflexion [56] works on the principle that LLMs can improve by analyzing their own past performance through natural language. First, the Actor generates a trajectory, which is essentially a sequence of thoughts and actions intended to solve a given task. Once the action is taken, the Evaluator assesses the resulting output by using mechanisms such as heuristics, unit tests, or another LLM, ultimately providing a score or specific feedback based on performance. In instances where the task fails, the system triggers the Self-Reflection module, where an LLM analyzes the

trajectory and the feedback to write a verbal "lesson learned." This reflection is then stored in the agent's Memory, where it is retrieved and utilized by the Actor in the next trial, effectively guiding the agent toward the correct behavior through its accumulated experience.

Together, these components transform LLMs from passive generators into autonomous systems capable of reasoning, interacting with external environments, retaining experience, and improving their behavior over time. This transition provides the foundation for more advanced architectures, including multi-agent systems and agent-based Decision Support Systems, where multiple specialized components collaborate to solve complex tasks.

2.13 Modern LLM-based Decision Support Systems

The potential for LLMs to transform decision-making is underscored by Eigner and Händler [16], who provide a comprehensive analysis of the determinants that govern successful human-AI collaboration. While traditional decision-making follows the classic intelligence-design-choice phases, the integration of LLMs introduces a far more complex set of variables. Their research highlights that effective support is not merely a technical challenge but a socio-technical one, involving a delicate interplay between technological aspects (such as prompt engineering and transparency), psychological factors (such as user mental models, trust, and decision-making styles), and decision-specific determinants (such as task difficulty and accountability). By mapping these interdependencies into a consolidated dependency framework, Eigner and Händler emphasize that improving decision quality requires moving beyond simple model performance to focus on the alignment between AI capabilities and human cognitive needs. The state-of-the-art in this domain has recently moved toward *adaptive, multi-agent architectures* that operationalize these reasoning frameworks within high-stakes environments. A primary example is the work of Öğdü et al. [43], who developed a multi-layered, adaptive Clinical Decision Support System (CDSS). Unlike traditional static models, this architecture integrates biomedical RAG with real-time web intelligence through a collaborative framework of autonomous agents. The system introduces several key innovations:

- **Timeliness and Evidence Freshness:** By incorporating a dedicated web-intelligence agent, the system mitigates the "knowledge cutoff" problem of static LLMs, ensuring recommendations are grounded in the latest clinical guidelines.
- **Confidence-Aware Optimization:** The system utilizes an adaptive feedback loop that triggers re-querying and hypothesis refinement whenever internal confidence scores fall below a defined threshold ($\tau = 0.65$).
- **Domain-Specific Grounding:** To ensure precision in medical contexts, the system employs a specialized BioBERT-based vector database for processing complex clinical jargon.

Validated on the MedQA and PubMedQA datasets—achieving accuracies of 94% and 88% respectively, this approach demonstrates how the integration of multi-source data fusion and adaptive optimization loops can overcome the reliability barriers of standard LLM-based reasoning.

Building upon this transition toward adaptive architectures, Alkayyal and Malberg introduce StrategicAI [2], a Decision Support System (DSS) tailored for strategic business contexts. Strategi-

cAI operationalizes the logic tree methodology to manage the high uncertainty and cognitive load inherent in strategic planning. The system functions through a three-phase multi-agent process:

- **Recursive Problem Decomposition:** Rather than providing a single output, the system uses a recursive, multi-agent architecture to decompose complex strategic problems into granular issue trees, mapping potential causes and solutions hierarchically.
- **Evidence-Based Retrieval Augmentation:** The system integrates a retrieval-augmented LLM (RAG) that draws from both internal organizational documentation and external market data. This ensures that the generated tree nodes are not merely speculative, but grounded in evidence-based business intelligence.
- **Human-in-the-loop Orchestration:** Addressing the socio-technical challenges identified by Eigner and Händler, StrategicAI maintains a collaborative workflow where the AI structures and validates the logic, while the human expert retains control over the strategic direction.

By automating the structural decomposition of business problems and mitigating cognitive biases through evidence-backed reasoning, StrategicAI demonstrates the utility of applying adaptive, agentic workflows to high-stakes management decisions where traditional static models often fail to account for complex, non-linear variables.

In the domain of urban governance, Kalyuzhnaya et al. [33] propose a multi-agent LLM framework designed to bridge the gap between strategic urban planning and operational execution. Moving beyond standalone model limitations, the authors introduce a modular architecture that integrates LLMs with existing municipal information systems to address the inherent fragmentation of urban data. The system operates through a hierarchical orchestration framework:

- **Hierarchical Orchestration Layer:** The architecture is centered on an "agent-conductor," which serves as the system's intelligent core. This agent is responsible for query intent classification and routing, ensuring that incoming requests are directed to the appropriate specialized processing pipelines.
- **Specialized Processing Agents:** The system utilizes three primary functional agents to handle complex urban queries:
  - *Tool Calling Agent:* Manages real-time data retrieval via APIs, providing access to transportation metrics, urban service availability, and geospatial layers.
  - *RAG Operations Agent:* Executes retrieval-augmented generation against a dedicated vector database, grounding the LLM's reasoning in validated regulatory documents and municipal development frameworks.
  - *Answer Summarization & Validation Agents:* Synthesizes retrieved data and validates the generated responses for consistency, reducing the risk of hallucinations often associated with non-grounded LLM implementations.
- **Integration with Digital Urban Platforms:** Unlike isolated decision models, the system maintains a bidirectional link with city information systems, such as the Digital Urban Platform of St. Petersburg. This allows the agents to process heterogeneous data—ranging

from socio-economic indicators to infrastructure capacity—enabling the system to provide context-aware strategic recommendations.

Unlike conventional business intelligence systems, StrategicAI does not only retrieve and summarize information but assists decision-makers in structuring ambiguous problems before evaluating possible alternatives.

While the urban governance framework focuses on synthesizing administrative data for high-level decision support, the technical principles of modular agentic architectures have also been validated in high-precision scientific domains. In a landmark study on autonomous research, researchers demonstrated an LLM-based system: Coscientist [7], that bridges the gap between high-level reasoning and physical execution in chemical laboratories. While the urban governance model prioritizes RAG operations for regulatory compliance, Coscientist emphasizes a closed-loop "tool-use" paradigm:

- **Action-Oriented Modularity:** The system decomposes complex research tasks into discrete commands (e.g., web search, code execution, hardware API interaction). This demonstrates that, when provided with a defined action space and sandboxed execution environments, LLMs can transition from passive information retrieval to active, self-correcting problem solving.
- **Grounding via Dynamic Documentation:** Beyond static RAG, Coscientist employs vector-based retrieval on technical documentation to dynamically learn and interact with APIs it was not explicitly trained to handle. This highlights the potential for decision support systems to maintain relevance in rapidly evolving technical landscapes without requiring constant model retraining.
- **Autonomous Self-Correction:** A significant contribution of this work to the field of decision support is the system's capacity for diagnostic iteration. When an generated protocol fails (e.g., an invalid hardware call), the agent autonomously queries its documentation module to rectify the error. This "closed-loop" reasoning is essential for moving toward autonomous systems capable of operating in high-stakes environments where human oversight may be intermittent.

By comparing these two domains—urban governance and experimental chemistry—it becomes clear that the utility of LLM-based decision support lies not in the model's inherent knowledge, but in its ability to operate as an intelligent orchestrator within a modular, tool-rich ecosystem. Unlike conventional business intelligence systems, both StrategicAI and the Coscientist framework shift the paradigm from retrieval to intervention, assisting decision-makers in structuring ambiguous problems and executing evidence-based alternatives in real-time.

To address the operational challenges of future deep-space exploration—specifically communication latency and the requirement for crew autonomy, the work of Kaisheng et al. [37] proposes an Earth-independent, agentic AI framework for Extravehicular Activity (EVA) mission planning. The system functions as a localized "virtual flight controller," integrating a fine-tuned Large Language Model (LLM) with a robust Retrieval-Augmented Generation (RAG) architecture and external physics-based simulation models.

The core innovation of the proposed system lies in its ability to synthesize heterogeneous data sources, ranging from formal flight rules and procedural handbooks to real-time suit telemetry and

metabolic predictive models, into actionable, traceable decision support. By utilizing an agentic orchestration layer, the system performs iterative, chain-of-thought reasoning that mimics the analytical rigor of ground-based Mission Control.

The decision support system provides critical utility in three primary domains:

- **Dynamic Replanning:** Enables astronauts to conduct "what-if" simulations, allowing for the rapid recalculation of mission timelines and resource margins in response to unexpected discoveries or environmental changes.
- **Off-Nominal Contingency Management:** Provides immediate, context-aware troubleshooting guidance during system failures, ensuring safety margins are continuously monitored even when connectivity to Earth is unavailable.
- **Cognitive Load Reduction:** Acts as an intelligent interface that abstracts complex sensor data and procedural constraints into concise, explainable recommendations, thereby allowing the crew to maintain high levels of operational awareness during high-stakes maneuvers.

By grounding LLM-generated outputs in verifiable physics models and authoritative mission documents, the architecture ensures high transparency and auditability. This framework serves as a foundational step toward increasing crew independence and operational safety for long-duration missions on the lunar surface and beyond.

Addressing the integration of Large Language Models into space-faring autonomy, Maranto [41] proposes LLMSAT: a hierarchical architecture that positions an LLM as a high-level mission operations planner, decoupled from low-level spacecraft control. To mitigate the inherent risks of non-deterministic LLM behavior, this architecture employs a 'correctness-by-construction' paradigm, where the agent interacts exclusively with validated flight software services. Safety is enforced through a tripartite validation framework: function-level argument checking, plan-level simulation-based validation, and reflexive spacecraft-level safety overrides. The implementation leverages Retrieval-Augmented Generation (RAG) to manage context-window constraints and facilitate experiential learning, demonstrating a shift toward agentic mission management while emphasizing the need for robust, ground-validated command interfaces in space-computing environments.

Despite these promising developments, current LLM-based DSS for mission operations remain largely domain-specific and prototype-oriented. Existing systems typically focus on individual operational scenarios, such as EVA planning or spacecraft autonomy, rather than providing a general-purpose, multi-agent decision support architecture capable of integrating heterogeneous operational data, procedural knowledge, human oversight, and coordinated reasoning. This gap motivates the research presented in this thesis, which investigates a multi-agent LLM-based Decision Support System tailored for space mission operations.

Chapter 3

System Engineering & Requirement Analysis

3.1 Domain and Mission Context

The ground segment of a space mission comprises all the facilities, software systems, communication infrastructure, and operational personnel responsible for supporting spacecraft operations throughout the mission lifecycle. It acts as the interface between the spacecraft and mission operators on Earth, enabling mission planning, spacecraft monitoring, command transmission, and anomaly management.

Within the Fucino Space Centre, the core ground segment is organized into several specialized subsystems, each responsible for a specific operational function. The main components include:

- **Satellite Control Center (SCC)**, responsible for spacecraft monitoring and command execution;
- **Mission Planning Center (MPC)**, responsible for planning operational activities and scheduling mission tasks;
- **Telemetry, Tracking and Control (TT&C)**, responsible for telemetry acquisition, spacecraft tracking, and command uplink;
- **Flight Dynamics**, responsible for orbit determination, maneuver planning, and trajectory analysis;
- **Mission Monitoring and Control (MMC)**, responsible for supervising mission execution and operational status;
- **Planning and Control Center (CPCM)**, including planning tools used to coordinate mission activities;
- **Communication Infrastructure (COMI)**, enabling data exchange between the different ground segment subsystems.

Although each subsystem performs a well-defined function, mission operations emerge from their continuous interaction. Operational data are generated by multiple heterogeneous sources and

distributed across independent software systems. Consequently, obtaining a complete and coherent picture of the mission state requires correlating information originating from several subsystems operating simultaneously.

This distributed operational environment makes the ground segment inherently information-intensive and places significant cognitive demands on mission operators responsible for supervising mission execution.

3.2 The Mission Manager's Role and Challenges

Among the different operational roles, the Mission Manager is responsible for supervising the overall execution of the mission and coordinating the activities of the operational teams. The Mission Manager maintains situational awareness of the mission, evaluates operational events, coordinates responses to anomalies, and supports decision-making during both nominal and contingency operations.

To perform these tasks, the Mission Manager must continuously integrate information coming from multiple sources, including spacecraft telemetry, mission planning systems, flight dynamics analyses, communication systems, operational procedures, and reports generated by different operators. These information sources are often heterogeneous, asynchronous, and, in some situations, may provide incomplete or even conflicting evidence regarding the current mission state.

The increasing complexity of modern space missions has significantly amplified the amount of information that operators must process in real time. During critical operational phases or unexpected events, rapidly identifying relevant information and understanding its implications becomes particularly challenging. Maintaining situational awareness under these conditions requires considerable expertise and imposes a substantial cognitive workload on the Mission Manager.

These challenges motivate the adoption of intelligent Decision Support Systems capable of aggregating information from heterogeneous sources, assisting operators in assessing the current operational context, and supporting timely and informed decision-making.

The system we are going to develop is called FUNES: Foresight Understanding for Nominal and Emergency Scenarios. The objective of FUNES is to support mission managers during operational decision-making by reducing information fragmentation, improving situational awareness, and decreasing the cognitive workload associated with complex mission scenarios.

3.3 FUNES System Overview

FUNES (Foresight Understanding for Nominal and Emergency Scenarios) is an AI-based Decision Support Tool designed to support mission operations within an Earth Observation ground segment environment. The system acts as an intermediate layer between existing mission operational systems and human operators, collecting heterogeneous operational data, analysing mission conditions, and providing decision-support outputs.

FUNES does not replace mission operators or execute autonomous spacecraft operations. Instead, it supports human decision-making by improving situational awareness, reducing information fragmentation, and providing recommendations based on available mission knowledge and operational data.

The main functions provided by FUNES include:

- acquisition and processing of heterogeneous mission data;
- detection of anomalies and operational degradations;
- correlation of information from different mission domains;
- generation of operational recommendations;
- provision of explainable AI outputs to support operator decisions.

The system operates within the existing ground segment infrastructure and interfaces with external mission systems, including telemetry sources, mission planning systems, flight dynamics systems, monitoring tools, and operator interfaces.

3.4 System and Software Requirement Specification according to ECSS

Following the ECSS requirement engineering approach—incorporating standard guidelines for system engineering [18], software development [19], and software product assurance [20]—the FUNES development process produced two main specification documents:

- **System Requirements Document (SRD):** defining the technical requirements of the FUNES system as a complete entity within the mission ground segment environment;
- **Software Requirements Specification (SRS):** defining the software-level requirements derived from the system specification and allocated to the software components.

The System Requirements Document represents the first level of technical specification. It describes the expected behaviour of the system independently from the implementation details and defines the capabilities required to satisfy operational needs. The SRD includes:

- **Functional requirements** describing the capabilities provided by the system;
- **performance requirements** defining measurable system constraints;
- **interface requirements** describing interactions with external and internal entities;
- **operational requirements** defining system behaviour during different operational modes;
- **safety, dependability, data management, and AI-specific requirements**.

According to ECSS principles, each requirement is written using a structured and verifiable formulation based on the "shall" statement. Requirements are required to be necessary, unambiguous, atomic, implementation-independent, verifiable, and traceable.

The Software Requirements Specification is derived from the system specification by allocating system functions to software components. It defines the expected behaviour of the software architecture, including software functionalities, internal interfaces, data processing requirements, AI module interactions, and verification criteria.

The relationship between the two documents follows a hierarchical requirement flow where system-level requirements are progressively refined into software-level requirements while maintaining bidirectional traceability through a Requirement Traceability Matrix (RTM).

3.5 System Engineering Approach

The development of the FUNES system followed a system engineering approach based on the principles defined by the ECSS. The objective of the system engineering process was to transform operational needs identified within the mission control environment into a structured and traceable set of system and software requirements.

The adopted approach followed a top-down requirement derivation process, starting from the analysis of the operational scenario and stakeholder expectations, and progressively refining them into functional, performance, interface, operational, and AI-specific requirements. The requirement definition process was performed according to the principles of ECSS-E-ST-10C, ensuring that each requirement was necessary, unambiguous, verifiable, traceable, atomic, and implementation-independent.

The identification of stakeholder needs represents the first step of the system engineering process. For the FUNES system, the primary stakeholders were identified within the mission operations environment, including Mission Managers, spacecraft operators, ground segment operators, and system maintenance personnel.

The analysis of the operational scenario highlighted several challenges associated with current mission operations. Mission operators must continuously monitor heterogeneous information sources, including spacecraft telemetry, mission planning information, ground segment status, flight dynamics data, network data, and operational reports.

During nominal and non-nominal operations, the integration and interpretation of such information require significant human effort, especially when rapid decisions are required. Therefore, the main operational need identified for FUNES was the capability to improve mission situational awareness by aggregating heterogeneous operational information and providing decision-support capabilities while preserving human authority over final operational decisions.

3.6 System Requirements Definition

The System Requirements Document (SRD) defines the technical requirements governing the functional behavior and operational constraints of the FUNES system.

Following the ECSS-E-ST-10C standard, a total of around 160 requirements were formulated to guarantee end-to-end traceability between operational needs and verification activities. The main requirements are:

- **Functional Requirements (FUN - 86 requirements):** Core system capabilities, including data acquisition, anomaly detection, recommendation generation, explainability, and human-in-the-loop interaction.
- **Performance Requirements (PER - 6 requirements):** Quantitative constraints on accuracy, latency, reliability, scalability, and resource consumption.

- **Interface Requirements (INT - 12 requirements):** External integrations with telemetry sources, mission planning systems, databases, and user interfaces.
- **Operational Requirements (OPR - 12 requirements):** System behavior during nominal, validation, training, and maintenance operations.
- **Non-functional Requirements (NFR - 13 requirements):** Cross-cutting quality attributes addressing system robustness, interface reliability, and operational constraints.
- **AI Requirements (AI - 42 requirements):** Requirements related to the Artificial Intelligence (AI) capabilities of the FUNES system, including model selection, data usage, explainability, bias mitigation, and lifecycle management.

3.7 Software Requirement Specification

The Software Requirements Specification (SRS) was derived from the System Requirements Document following the ECSS top-down requirement engineering process. While the SRD defines the expected behaviour of the system from an operational perspective, the SRS allocates these requirements to software components and specifies the software architecture required to implement them.

3.7.1 Functional Decomposition and Allocation

Starting from the system-level requirements defined in the SRD, the FUNES system was decomposed into a set of functional capabilities representing the main operational objectives of the Decision Support System.

Following the ECSS top-down requirement engineering approach, each capability was further refined into a set of lower-level system functions. These functions describe the internal behaviours required to implement the corresponding capability and provide the basis for the allocation of requirements to software Configuration Items.

The main capability-to-component allocation is summarized in Table 3.1.

C-01: Mission Data Ingestion Capability The system shall be capable of acquiring, normalizing, validating, and correlating heterogeneous mission data from multiple external sources in near real-time. This capability is decomposed in the following set of functions:

- **F-01 Data acquisition from external systems:** Establishes and manages secure communication channels with external sources to ingest mission data streams.
- **F-02 Data format normalization and harmonization:** Converts heterogeneous incoming data structures into a unified, standardized internal data model.
- **F-03 Data validation and quality assessment:** Inspects ingested data against predefined syntactic and semantic rules to filter out corrupted or non-compliant payloads.
- **F-04 Timestamping and cross-source correlation:** Appends synchronized system timestamps and correlates data packages sharing spatial, temporal, or logical relationships.
- **F-05 Secure ingestion and integrity verification:** Cryptographically verifies data origin and integrity (e.g., via checksums/signatures) to prevent tampering during ingestion.

C-02: AI-based Analysis and Decision Support Capability The system shall be capable of performing AI-driven analysis of mission data to detect anomalies, assess system performance, and generate operational recommendations with associated confidence and explainability information. This capability is decomposed in the following set of functions:

- **F-06 Anomaly detection inference:** Evaluates data streams to identify deviations from nominal events.
- **F-07 Trend and pattern analysis:** Processes historical and real-time data to identify long-term behaviors, recurring operational patterns, and performance degradation.
- **F-08 Operational recommendation generation:** Translates analytical insights and detected anomalies into actionable mitigation strategies or operational decisions for the operator.
- **F-09 Confidence scoring of analytical outputs:** Computes a probabilistic metric representing the certainty level of each detected anomaly and generated recommendation.
- **F-10 Explainability artifact generation:** Produces human-readable justifications and feature-importance metrics to explain the rationale behind AI-driven outputs.

C-03: Mission Status Visualization and Operator Interaction Capability The system shall be capable of providing mission status visualization, including alerts, recommendations, and operational reports, and supporting operator interaction and feedback collection. This capability is decomposed in the following set of functions:

- **F-11 Conversational Interface:** Handles the conversation with the LLM-based system.
- **F-12 Mission status dashboard generation:** Renders the primary user interface, displaying real-time system metrics, operational KPIs, and overall mission health.
- **F-13 Alert and anomaly visualization:** Highlights detected anomalies and system alerts using visual hierarchies, sound cues, and priority levels to capture operator attention.
- **F-14 Recommendation presentation and prioritization:** Displays generated operational recommendations, sorted by confidence score, priority, and critical urgency.
- **F-15 Operational reporting and historical view rendering:** Compiles standard operational reports and allows the operator to query, filter, and view historical telemetry and past events.
- **F-16 Operator interaction and feedback handling:** Captures explicit operator inputs, acknowledgments of alerts, and qualitative feedback on recommendations, routing them to relevant subsystems.

C-04: Configuration and Model Lifecycle Management Capability The system shall be capable of managing system configuration, AI model lifecycle, versioning, deployment, and retraining workflows under controlled operational governance. This capability is decomposed in the following set of functions:

- **F-17 System configuration management:** Controls, tracks, and validates system-wide operational parameters, thresholds, and environmental settings.
- **F-18 AI model versioning and deployment control:** Manages the repository of trained AI models, overseeing version control, seamless deployment to production, and rollback capabilities.
- **F-19 Dataset versioning and metadata management:** Tracks and catalogues training, validation, and testing datasets, preserving metadata regarding data lineage and provenance.
- **F-20 Adaptation trigger definition and orchestration:** Monitors model performance metrics and system-defined thresholds to identify conditions requiring model retraining or adaptation and orchestrates the corresponding workflow triggers.
- **F-21 System rollback and restore point management:** Generates system state snapshots and coordinates rollbacks to previous stable configurations in the event of deployment failures.

C-05: Persistent Storage and Historical Retrieval Capability The system shall be capable of persistently storing operational data, analytical artifacts, configuration snapshots, and historical records, and providing controlled retrieval interfaces to authorized components. This capability is decomposed in the following set of functions:

- **F-22 Persistent data storage:** Manages the secure, long-term physical or logical storage of all incoming data, normalized records, and system states.
- **F-23 Historical data retrieval:** Exposes high-performance querying and indexing APIs to allow system components to fetch historical records.
- **F-24 Audit log persistence:** Records an unalterable sequential log of all user actions, system state transitions, and security-relevant events for compliance.
- **F-25 Data integrity and consistency enforcement:** Executes periodic checks, replication routines, and database constraints to guarantee data consistency and prevent corruption.
- **F-26 Data retention policy enforcement:** Automatically archives or purges older data records according to predefined regulatory or operational storage lifecycle policies.

3.7.2 Software Architecture

The software architecture of the FUNES system was derived through a progressive refinement process starting from the operational needs identified during the system engineering phase and the requirements defined in the System Requirements Document (SRD).

Following the ECSS top-down requirement engineering approach, system-level capabilities were progressively decomposed into system functions and allocated to dedicated software components. This process allowed the transition from an operational view of the system, focused on mission manager needs and decision-support capabilities, to a software-oriented view defining the internal organization of the FUNES platform.

Table 3.1: Capability-Function Traceability Summary

Capability ID	Capability	Functions
C-01	Mission Data Ingestion Capability	F-01, F-02, F-03, F-04, F-05
C-02	AI-based Analysis and Decision Support Capability	F-06, F-07, F-08, F-09, F-10
C-03	Mission Status Visualization and Operator Interaction Capability	F-11, F-12, F-13, F-14, F-15, F-16
C-04	Configuration and Model Lifecycle Management Capability	F-17, F-18, F-19, F-20, F-21
C-05	Persistent Storage and Historical Retrieval Capability	F-22, F-23, F-24, F-25, F-26

The main architectural driver was the need to support heterogeneous mission data processing, AI-based analysis, operator interaction, configuration control, and historical data management while maintaining modularity, scalability, and verification independence.

Starting from the system requirements, the main functional capabilities were decomposed and allocated to five independent **Configuration Items (CIs)**, each representing a logically separated software subsystem responsible for a specific functional domain (Fig. 3.1).

Configuration Items Breakdown

Below is the detailed functional decomposition and allocation for each Configuration Item defined in the System Requirements Specification (SRS).

CI-1 – Data Processing Manager (DPM) The Data Processing Manager is responsible for data ingestion, format normalization, quality validation, and spatio-temporal correlation of heterogeneous mission information coming from multiple space and ground segment sources.

- **Inputs:** Spacecraft telemetry, operational status, Mission Planning System (MPS) schedules, Ground Station & antenna status, Payload Data Ground Segment (PDGS) outputs, Flight Dynamics System (FDS) orbit/attitude data, infrastructure/network telemetry, and ticketing incident data.
- **Outputs:** Normalized data records formatted according to the Unified Internal Data Model
- **Key Responsibilities:**
  - Multi-source data acquisition via standardized Interface Control Documents (ICDs).
  - Timestamp assignment, cross-source correlation, and cryptographic integrity check.
  - Isolation of faulty or corrupted data streams to ensure system fault tolerance.
  - Persistence of normalized records to CI-5.
- **Allocated Functions:** `F-01, F-02, F-03, F-04, F-05`.

CI-2 – AI Analytics Manager (AIM) The AI Analytics Manager executes AI-driven analytical processing using an ensemble of neural models, deterministic processing rules, and tool-assisted workflows to evaluate anomalies and generate actionable operational guidance.

- **Inputs:** Normalized and correlated data records from CI-1; historical context data from CI-5.
- **Outputs:** Structured analytical artifacts containing: `anomaly_flag`, `confidence_score`, `affected_system`, `recommended_action`, `model_version`, and `explainability_reference`.
- **Key Responsibilities:**
  - Stream and batch execution of AI model inference (anomaly detection, trend analysis, root cause analysis).
  - Recommendation generation, prioritization, and explainability artifact generation (XAI).
  - AI model performance monitoring, bias detection, and operator feedback integration.
- **Allocated Functions:** `F-06, F-07, F-08, F-09, F-10`.

  Note: All analytical outputs generated by CI-2 are explicitly designated as advisory. Operator evaluation is strictly required prior to operational application.

CI-3 – Decision Support Interface (DSI / User Interface) The Decision Support Interface provides operator-facing dashboards and interaction capabilities for real-time mission monitoring, situation awareness, and decision support.

- **Inputs:** Real-time alerts, rankings, recommendations, and XAI artifacts from CI-2; historical reports from CI-5; configuration status from CI-4.
- **Outputs:** Operator queries, recommendation feedback (commit, reject, modify, rate), operational reports, and configuration requests.
- **Key Responsibilities:**
  - Interactive dashboard rendering for alerts and recommendations.
  - Authentication, Role-Based Access Control (RBAC), and Multi-Factor Authentication (MFA).
  - Displaying system state transitions, safe-state entry, and retraining notifications.
- **Allocated Functions:** `F-12, F-13, F-14, F-15, F-16`.

CI-4 – Configuration & Model Management (CMM) This component governs the global configuration parameters of the FUNES system, AI model lifecycles, prompt/dataset versioning, and deployment operations.

- **Inputs:** Configuration requests from operators (via CI-3); model performance degraded-metrics and retraining triggers from CI-2.
- **Outputs:** System configuration updates, model versioning commands, and rollback procedures.

- **Key Responsibilities:**
  - Centralized configuration parameters and operational mode state machine management.
  - AI model repository control, prompt engineering versioning, and fine-tuning dataset management.
  - Orchestrating model retraining workflows and creating pre-update restore points.
- **Allocated Functions:** `F-17, F-18, F-19, F-20, F-21`.

CI-5 – Storage Manager (SM) The Storage Manager provides centralized, secure, and high-performance persistent storage services for all system components.

- **Inputs:** Raw and normalized records, analytical artifacts, audit logs, and configuration snapshots from CI-1, CI-2, CI-3, and CI-4.
- **Outputs:** Historical datasets, retrieved records, system configuration snapshots, and audit logs.
- **Key Responsibilities:**
  - Providing continuous storage and retrieval APIs for all CIs.
  - Enforcing data retention, consistency, and integrity policies across logical data stores.
- **Allocated Functions:** `F-22, F-23, F-24, F-25, F-26`.

External System Interfaces

To operate effectively within the ground segment ecosystem, FUNES establishes continuous interfaces with several external systems:

- **Spacecraft Telemetry Systems:** Telemetry stream ingestion (HKTM).
- **Mission Planning System (MPS):** Timelines, scheduled operational events, and plan execution states.
- **Ground Segment & Antenna Monitoring:** Ground station health status and RF link metrics.
- **Flight Dynamics System (FDS):** Orbital ephemerides, attitude state vectors, and maneuver plans.
- **Ticketing & Incident Management:** Operational anomaly records and maintenance logs.

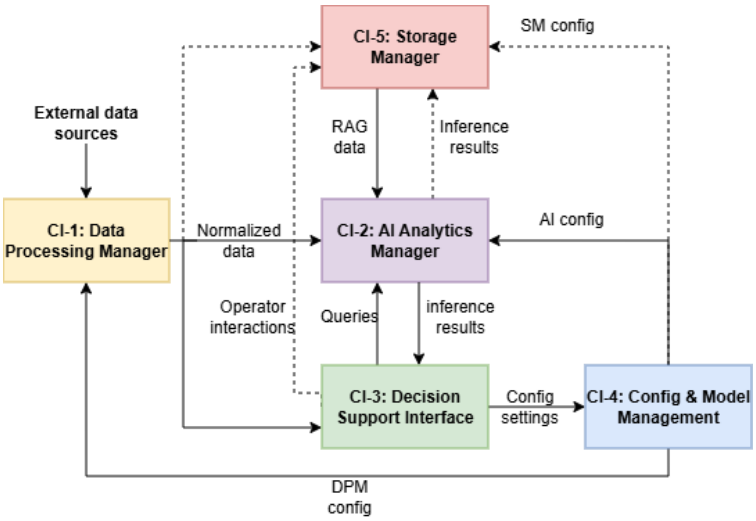

Figure 3.1: The Funes high level software architecture is composed of 5 Configuration Items. The data flows from external subsystems to the Data Processing Manager (DPM) module which will turn them into an internal standard data representation after performing cleaning and validation. The data is then fed to the Storage Manager (SM) for beign stored and to the AI Analytics for AI processing. The user utilizes the system through the Decision Support Interface (DSI). The Config & model management module is responsible for the system configuration and AI control.

3.7.3 Software Requirements Derivation

Following the functional decomposition presented in the previous section, the Software Requirements Specification (SRS) was obtained through the progressive refinement of the system requirements defined in the System Requirements Document (SRD).

In accordance with the ECSS top-down requirement engineering methodology, each system requirement was first allocated to one or more functional capabilities and subsequently decomposed into lower-level system functions. These functions were then further refined into a set of atomic software requirements, each describing a single software behaviour that can be independently implemented, verified, and traced throughout the software lifecycle.

The decomposition process follows the principle of *requirement atomicity*. Whenever a system requirement specifies multiple independent behaviours, it is split into several software requirements, each addressing only one responsibility. This refinement improves requirement clarity, facilitates implementation, and enables independent verification of every software function.

For example, the system requirement

> *FUNES-AI-12.1-003: "The FUNES system shall support semantic retrieval of operational records, historical decisions, and mission outcomes for AI-assisted reasoning processes."*

contains three distinct retrieval capabilities. During the software specification phase, it is decomposed into three independent software requirements:

- **SRS-AI-005**: The software shall support semantic retrieval of operational records for AI-assisted reasoning.
- **SRS-AI-006**: The software shall support semantic retrieval of historical decisions for AI-assisted reasoning.

- **SRS-AI-007**: The software shall support semantic retrieval of mission outcomes for AI-assisted reasoning.

Although these software requirements originate from the same parent system requirement, each one can be independently allocated to software components, implemented, and verified.

Every software requirement maintains bidirectional traceability with its originating system requirement while being assigned to the Configuration Item responsible for its implementation. Furthermore, following the ECSS verification philosophy, each software requirement is associated with a verification method selected among Test (T), Analysis (A), Inspection (I), or Review of Design (RoD). This information establishes a complete traceability chain from the original operational requirement to the corresponding implementation and verification activities.

The adopted refinement process resulted in a Software Requirements Specification comprising **358 atomic software requirements**. Each requirement is uniquely identified, allocated to a specific Configuration Item, linked to its parent system requirement, and associated with the corresponding ECSS verification method.

The resulting traceability chain adopted throughout the development process is illustrated below:

$$\begin{aligned}\text{System Requirement} \rightarrow \text{System Function} \rightarrow \text{Atomic Software Requirement} \\ \rightarrow \text{Configuration Item} \rightarrow \text{Verification Method}\end{aligned}$$

This hierarchical organization guarantees complete traceability between operational needs, software implementation, and verification activities, while ensuring compliance with the model-based systems engineering process adopted for the development of the FUNES platform.

3.7.4 Software Design

The software design phase refines the architectural decomposition introduced in the previous sections by defining the implementation technologies, software frameworks, and deployment strategies adopted for the FUNES system.

The technology selection was driven by the software requirements identified in the Software Requirements Specification (SRS), with particular attention to modularity, scalability, maintainability, interoperability with existing ground segment infrastructures, and suitability for AI-based mission support applications.

The system is primarily implemented using Python as the common development language across all Configuration Items. Python was selected due to its mature ecosystem for artificial intelligence, machine learning, data processing, and scientific computing, as well as its widespread adoption in data-intensive software applications. The use of a common development environment simplifies integration between traditional software modules and AI-based components.

Where strict performance constraints may arise, particularly in high-throughput data processing tasks, lower-level implementations through C++ or Rust extensions can be introduced at module level without affecting the overall software architecture.

The implementation of the individual Configuration Items is described below.

CI-1 - Data Processing Manager

The Data Processing Manager is responsible for the acquisition, ingestion, normalization, validation, and correlation of heterogeneous mission data sources.

To support continuous data ingestion and asynchronous processing, CI-1 relies on Apache Kafka as the underlying event-streaming platform. Kafka provides fault-tolerant message handling, ordered data delivery, and scalable processing capabilities suitable for heterogeneous mission data streams.

The incoming data are validated and transformed into the Unified Internal Data Model before being distributed to the other system components. Data validation and schema enforcement are implemented using Pydantic, enabling structured validation rules and consistency checks before data propagation.

The design of CI-1 supports integration with multiple external mission systems, including telemetry providers, mission planning systems, flight dynamics systems, and ground infrastructure monitoring systems.

CI-2 - AI Analytics Manager

The AI Analytics Manager represents the core analytical component of FUNES and is responsible for anomaly detection, trend analysis, operational recommendation generation, and explainability artifact production.

The component is based on a self-hosted Large Language Model (LLM) inference architecture, selected to support deployment within controlled mission environments where data sovereignty, operational security, and predictable performance are required.

LLM inference is performed through vLLM, which provides optimized high-throughput serving of open-weight models compatible with the HuggingFace model ecosystem. Candidate model families considered for deployment include LLaMA, Qwen, and Mistral, selected due to their open-weight availability and suitability for on-premise execution.

The orchestration of complex analytical workflows, including retrieval-augmented generation (RAG), tool-assisted reasoning, and multi-step agent workflows, is implemented using frameworks such as LangGraph and LlamaIndex.

In addition to generative AI capabilities, CI-2 integrates classical machine learning components for anomaly detection and trend analysis. These models are implemented using established frameworks such as Scikit-learn and PyTorch.

The AI outputs generated by CI-2, including anomaly assessments, confidence scores, recommendations, and explanations, remain advisory and are provided to support, rather than replace, mission operator decision-making.

CI-3 - Decision Support Interface

The Decision Support Interface provides the interaction layer between the FUNES system and mission operators.

The interface is implemented as a web-based application using React for the frontend and FastAPI as the backend service layer. This architecture enables the development of modular user interface components while maintaining efficient communication with the internal FUNES services.

Real-time updates, including alerts, dashboard information, and analytical results, are delivered through WebSocket communication channels, enabling continuous monitoring of mission conditions.

Authentication and authorization mechanisms are implemented through Keycloak, providing role-based access control and supporting multiple authentication methods, including username/password authentication, multi-factor authentication, and certificate-based authentication.

The interface provides mission dashboards, conversational interaction with the AI system, visualization of anomalies and recommendations, and mechanisms for collecting operator feedback.

CI-4 - Configuration and Model Management

The Configuration and Model Management component provides centralized control of system parameters, AI model lifecycle management, and configuration versioning.

The component uses MLflow to manage AI model versions, experiment tracking, deployment workflows, and model artifact registration. Dataset versioning and metadata management are also supported to preserve information regarding data provenance, lifecycle, and usage.

CI-4 manages model deployment, rollback procedures, retraining workflows, and configuration changes while maintaining reproducibility and traceability of system states.

These capabilities support the requirements related to AI lifecycle management and operational governance defined in the SRS.

CI-5 - Storage Manager

The Storage Manager provides persistent storage capabilities and standardized data access services for the other FUNES components.

The storage architecture follows a multi-store approach, where different types of information are stored according to their operational characteristics.

Time-series operational data, including normalized telemetry and ingestion records, are stored using TimescaleDB, a PostgreSQL extension optimized for time-dependent data workloads.

AI-related artifacts, including model files, datasets, and explainability outputs, are stored within an S3-compatible object storage system. This approach enables flexible deployment both in on-premise environments through solutions such as MinIO and in cloud-based infrastructures through compatible object storage services.

Audit logs and immutable operational records are maintained through PostgreSQL storage with append-only policies to preserve historical traceability.

Semantic embeddings generated for retrieval-augmented generation workflows are stored using vector indexing capabilities provided by pgvector, enabling similarity-based retrieval of historical mission data, operational procedures, anomaly reports, and previously generated analytical artifacts.

All access to persistent data is mediated through standardized storage interfaces, with encrypted communication channels and access control mechanisms applied at database and object storage levels.

Deployment and Integration Environment

The FUNES system is designed for deployment within hybrid cloud environments using containerized software components.

Docker is adopted for software packaging and environment isolation, while Kubernetes is used for service orchestration, scalability management, health monitoring, and deployment automation.

Communication between components follows a hybrid approach. Synchronous interactions are implemented through gRPC-based interfaces, while asynchronous information exchange relies on Kafka messaging mechanisms.

This deployment strategy enables flexible allocation of computational resources, particularly for AI-intensive workloads, while maintaining the modularity and operational robustness required for mission support applications.

3.7.5 System Data Flows and Operational Use Cases

This section describes the dynamic data flows of the FUNES system through representative operational use cases. Each use case illustrates the path data follows across the Configuration Items (CI-1 to CI-5), the transformations applied, and the interaction points where human-in-the-loop validation is required. Four primary use cases are defined to cover nominal automatic operations, interactive investigations, model maintenance, and degraded operational conditions.

UC-1: Near Real-Time Anomaly Detection and Recommendation (Nominal, Fully Automatic) This represents the primary nominal end-to-end data processing pipeline. Data arrives continuously from external space and ground sources, gets processed, analyzed, and presented to the operator. No operator intervention is required during the ingestion and analysis phases; human interaction is strictly required only at the final recommendation review and execution stage, adhering to the system's advisory-only design philosophy. See Fig. 3.2

1. **Data Acquisition & Normalization (CI-1):** External data streams (telemetry, MPS schedules, network logs) are ingested by CI-1 via dedicated ICD adapters (F-01). CI-1 normalizes payloads into the Unified Internal Data Model (F-02), validates syntax/semantics (F-03), appends system timestamps, and correlates records (F-04).

2. **Persistence (CI-5):** Normalized streams are continuously written to CI-5 for time-series persistence (F-22).

3. **AI Inference & Detection (CI-2):** Simultaneously, CI-1 emits normalized event streams to CI-2. CI-2 executes real-time anomaly detection inference (F-06). Upon detecting an anomaly, CI-2 generates an operational recommendation (F-08), calculates confidence scores (F-09), and produces explainability artifacts (F-10).

4. **Visualization & Notification (CI-3):** CI-2 transmits structured anomaly payloads to CI-3, which triggers visual/auditory alerts on the operator dashboard (F-13) and presents the prioritized recommendation (F-14).

5. **Operator Feedback & Decision (CI-3):** The operator evaluates the recommendation and explainability details. The operator explicitly commits, modifies, or rejects the suggestion (F-16). The feedback action is stored in CI-5 for compliance and future model optimization (F-24).

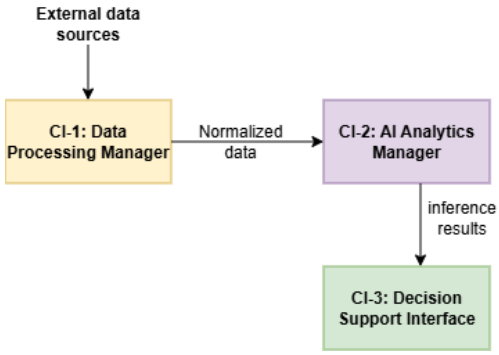

Figure 3.2: Data flow of use case 1

UC-2: Operator Conversational Investigation via AI-Assisted Chatbot This use case covers interactive decision support. The operator uses a natural language interface embedded in the dashboard to perform ad-hoc diagnostic queries, explore root causes, or analyze complex operational scenarios. See Fig. 3.3

1. **Query Submission (CI-3):** The operator formulates a free-text prompt within the conversational interface of CI-3 (F-11). CI-3 attaches session tokens, operator identity (RBAC), and forwards the request to CI-2.

2. **Agentic Workflow & Tool Execution (CI-2):** CI-2 parses the prompt using an LLM-based agent. To construct an accurate answer, CI-2 dynamically queries CI-5 via historical retrieval APIs (F-23) to fetch historical telemetry, past anomalies, or configuration snapshots.

3. **Response Generation & Grounding (CI-2):** CI-2 executes Retrieval-Augmented Generation (RAG), synthesize retrieved telemetry data into a structured response, and appends confidence scores (F-09) and source citations (F-10).

4. **Rendering & Multi-turn Dialogue (CI-3):** CI-3 renders the response, supporting interactive tables or chart widgets (F-11, F-15). The conversation may continue across multiple dialogue turns.

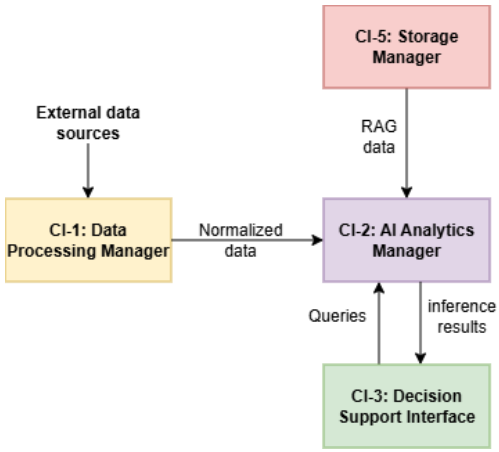

Figure 3.3: Data flow of use case 2

UC-3: Operator-Initiated Dashboard Investigation This use case describes a scenario where an operator observes a suspicious trend or receives an external notice (e.g., ground station maintenance) and manually triggers a deep-dive investigation across monitoring dashboards. See Fig. 3.4

1. **Manual Trigger & Filtering (CI-3):** The operator accesses the historical and real-time visualization views in CI-3 (F-12, F-15) and applies custom time-range, satellite, or ground station filters.

2. **Data Fetching (CI-5):** CI-3 issues high-performance query requests to CI-5, fetching raw/normalized historical parameters (F-23).

3. **On-Demand Analysis Trigger (CI-2):** The operator selects a telemetry window and requests an on-demand pattern/trend analysis (F-07). CI-2 processes the designated subset and returns comparative metrics or degradation curves to CI-3.

4. **Audit Logging (CI-5):** All query filters, manual inspection views, and operator actions are recorded in the audit log (F-24).

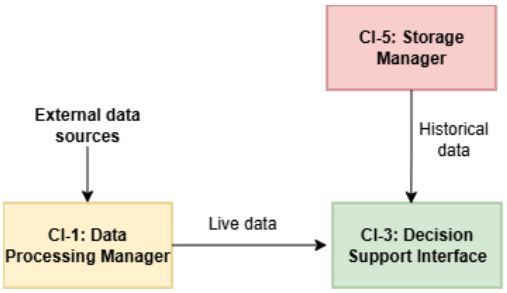

Figure 3.4: Data flow of use case 3

Chapter 4

Proof of Concept Development

4.1 PoC Overview and Architecture

The development of the Proof of Concept (PoC) followed the software architecture introduced in Section 3.7.2. The implementation was carried out following a bottom-up approach, progressively developing the core services required by the final decision-support system.

The proposed PoC is composed of three main software layers:

- the *Storage Manager*, responsible for providing unified access to heterogeneous information sources and managing the knowledge required by the system;
- the *AI Manager*, responsible for integrating Large Language Models, specialized agents, and the reasoning capabilities required to support decision-making activities;
- the *Decision Support Interface*, providing the interaction layer between users and the underlying AI-based services.

The communication and coordination between these layers are managed through the multi-agent orchestration framework described in the following sections.

Throughout the implementation, particular attention was devoted to the creation of a modular and extensible software architecture. Each component was developed following an abstraction-oriented design, where functionalities are exposed through well-defined interfaces and concrete implementations are decoupled from the higher-level logic.

This approach enables individual components to be replaced or extended with minimal modifications to the overall system. For example, different storage technologies, language models, or agent implementations can be integrated by providing alternative implementations of the corresponding interfaces.

The first component developed was the *Storage Manager*, as reliable access to contextual information represents a fundamental requirement for the subsequent reasoning and decision-support layers. Its implementation provides the data foundation of the system, including document retrieval capabilities through a custom Retrieval-Augmented Generation (RAG) pipeline, structured operational data access, and persistent conversation management.

4.2 Storage Manager

4.2.1 Architecture and Design Principles

The Storage Manager represents the data management layer of the proposed multi-agent framework. Its main responsibility is to provide a unified access interface to the heterogeneous information required by the AI components during the decision-support process.

Instead of allowing the reasoning layer to directly interact with individual data sources, the Storage Manager abstracts the underlying storage mechanisms and data formats. This design choice separates data management concerns from reasoning capabilities, reducing the dependency between the AI components and the specific implementation of the information sources.

The Storage Manager integrates multiple functionalities:

- a Retrieval-Augmented Generation (RAG) subsystem for managing technical documentation and enabling semantic knowledge retrieval;
- structured data interfaces for accessing operational information, including planning data stored in CSV, XML, and JSON formats;
- a persistent chat storage service for maintaining conversation histories between users and agents;
- additional interfaces for retrieving system status information and maintenance-related data.

Among these functionalities, the RAG subsystem represents the main knowledge management component of the Storage Manager. While structured data interfaces provide deterministic access to operational information, the RAG pipeline enables semantic exploration of unstructured technical documentation, supporting the reasoning capabilities of the multi-agent system.

For this reason, the following sections provide a detailed description of the RAG architecture, including document processing, embedding generation, vector storage, hybrid retrieval, and validation activities.

Following the same architectural principles adopted for the overall PoC, the Storage Manager has been implemented through an abstraction-oriented design. Core functionalities are defined through abstract interfaces and specialized through concrete implementations, allowing individual components to be replaced independently without affecting the remaining layers of the system.

This modular structure improves maintainability, supports experimentation with alternative technologies, and provides a scalable foundation for future extensions of the decision-support framework.

4.2.2 Rag System

Figure 4.2 illustrates the overall architecture of the implemented RAG in the Storage Manager.

The document ingestion process starts from the `Chunker`, responsible for loading heterogeneous documents and dividing them into semantically coherent chunks. The generated chunks are then converted into dense vector representations through the selected embedding model.

The resulting embeddings, together with their metadata, are stored inside the `Vector Store`, while a parallel lexical index based on `BM25` is maintained to support keyword-based retrieval.

Finally, the `Memory` component orchestrates the interaction among all the previous modules by managing document ingestion, preventing duplicate insertions, performing hybrid retrieval, and applying an additional reranking stage before returning the final context to the upper layers of the multi-agent framework.

Document Chunking

The first stage of the Storage Manager is responsible for transforming raw documents into semantically meaningful text fragments that can later be embedded and indexed. Since the quality of the retrieved information strongly depends on how documents are partitioned, particular attention was devoted to the design of the chunking process.

To ensure modularity, the chunking functionality is defined through the abstract class `BaseChunker`, which specifies the two fundamental operations required by the framework: loading a document and generating its corresponding chunks. This abstraction decouples the retrieval pipeline from any specific chunking strategy, allowing alternative implementations to be introduced without modifying the remaining components of the Storage Manager.

The concrete implementation developed for the Proof of Concept, named `LocalSemanticChunker`, performs semantic chunking locally by exploiting the *all-MiniLM-L6-v2* Sentence Transformer model. Instead of dividing documents into fixed-size windows, the adopted strategy identifies semantically coherent boundaries between consecutive text segments. This approach produces chunks that better preserve the original context, improving the quality of the subsequent embedding generation and retrieval phases. The selected Sentence Transformer model produces dense embeddings of 384 dimensions, providing a favorable trade-off between computational efficiency and semantic representation quality. The model was selected because it provides a favorable trade-off between embedding quality and computational cost, allowing all embeddings to be generated locally without requiring external APIs.

The semantic chunking process is implemented using the LangChain SemanticChunker with the `standard_deviation` breakpoint strategy. A breakpoint threshold of 1.5 is adopted, producing larger semantic segments by splitting only where the semantic distance between consecutive sentences significantly exceeds the average. Additionally, a buffer size of five sentences is used to avoid generating excessively small chunks. Moreover, chunks containing fewer than 50 characters or less than eight words are discarded, as they may provide insufficient contextual information and may negatively affect retrieval quality.

The implemented pipeline consists of several preprocessing steps before the actual semantic segmentation is performed. First, the input document is parsed according to its format (PDF, DOCX, or TXT). For PDF documents, metadata such as title, author, and last modification date are extracted and associated with every generated chunk.

To minimize noise and prevent non-informative repetitive text (such as running headers, footers, page numbers, and recurrent table structures) from polluting the vector space, a dynamic multi-stage text cleaning algorithm was designed and integrated prior to semantic chunking.

Unlike traditional static rule-based cleaning methods (e.g., rigid regular expressions), the proposed approach dynamically identifies document-specific boilerplate text through fuzzy string matching across page boundaries. The cleaning pipeline consists of three core steps:

1. **Dynamic Pattern Discovery via Sampling:** To optimize computational efficiency—especially for large technical documents, the algorithm performs an initial sampling over the first N pages ($N = 5$ by default). Each sampled page is decomposed into discrete line blocks.

2. **Cross-Page Fuzzy Similarity Analysis:** For every line block b_i in page i, the algorithm compares its content against all line blocks b_j in other sampled pages $j \neq i$. To account for minor textual variations (e.g., page numbers changing within a running header), textual overlap is computed using the matching algorithm via Python's `SequenceMatcher`.

   A line block is identified as a repetitive candidate pattern P_{rep} if its similarity ratio equals or exceeds a tuned threshold $\theta_{sim} = 0.95$:

   $$\text{similar}(a, b) = \frac{2 \cdot |M|}{|a| + |b|} \geq \theta_{sim} \tag{4.1}$$

   where $|M|$ is the number of matching characters in the longest common substrings between a and b, and $|a|, |b|$ are the lengths of the respective string blocks. Short lines (fewer than 10 characters) are ignored during pattern discovery to avoid accidentally stripping valid titles or bullet points.

3. **Document Filtering and Sanitization:** The discovered set of repetitive patterns P_{rep} is then used to filter the entire document page-by-page. Lines exceeding the similarity threshold when compared against any pattern in P_{rep} are pruned. Finally, string normalizations, including whitespace collapsing and low-character noise removal (discarding chunks under 50 characters or 8 words), are applied to produce clean, semantically dense text blocks.

This adaptive cleaning process ensures that recurring administrative header/footer metadata is stripped out, preventing artificial semantic similarity spikes during embedding generation and improving the signal-to-noise ratio in the retrieved context.

To preserve document structures during ingestion, PDF documents are parsed into structured Markdown format using `pymupdf4llm`. Unlike plain-text extractions that flatten layout elements, converting pages into Markdown preserves section headings, bullet lists, and tabular structures. Preserving tables and hierarchical layout in Markdown enables downstream embedding models to capture structural relationships, while remaining natively compatible with Large Language Model context processing.

After the cleaning phase, each page is semantically partitioned into coherent text fragments using the Semantic Chunker algorithm. Finally, each generated chunk is enriched with a comprehensive set of metadata, including a unique chunk identifier, a content hash made with SHA-1 computed over its normalized textual content used for duplicate detection, the source document, page number, chunk position within the page, chunk size, and document metadata. This information is preserved throughout the entire retrieval pipeline and enables efficient indexing, traceability, and document management.

The chunking process is showed in Fig. 4.1.

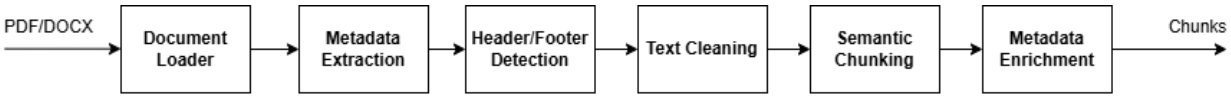

Figure 4.1: Document chunking workflow implemented in the Storage Manager. The document is preprocessed for metadata extraction like: "chunk_id", "content_hash", "source", "title", "author", "last_modified", "page_number", "chunk_index_in_page", "chunk_size" and "format". Then the repetitive parrterns like page headers and footers are removed. The text is then cleaned and transformed into markdown. Finally the chunking process will turn the text into vectors that are going to be associated with the metadata and the original text.

Embedding Generation

After the document chunking phase, each generated text fragment is transformed into a numerical representation suitable for semantic retrieval. This process is performed by the embedding generation module, whose objective is to map textual information into a continuous vector space where semantically similar contents are represented by nearby vectors.

Similarly to the other components of the Storage Manager, the embedding generation layer was designed following an abstraction-based approach. The abstract class `EmbeddingModel` defines the interface required by the system, exposing a single operation responsible for converting a text input into its corresponding embedding vector.

This abstraction decouples the retrieval pipeline from the specific embedding model used during execution. Therefore, different embedding techniques can be integrated by implementing the same interface, without requiring modifications to the memory management or storage layers.

For the Proof of Concept, the selected implementation is based on the Sentence Transformer architecture through the `all-MiniLM-L6-v2` model. The model generates dense vector representations that capture semantic relationships between textual fragments, enabling similarity-based retrieval even when the query and the stored document use different wording.

The generated embeddings are computed during the document ingestion phase and stored together with the corresponding text chunks and metadata inside the vector database. During the retrieval phase, the same embedding model is used to encode the user query, allowing the system to perform semantic similarity comparison between the query representation and the indexed knowledge base.

The separation between embedding generation and storage management allows the system to remain independent from a specific embedding technology, facilitating future experimentation with alternative models or domain-specific embedding approaches.

Vector Storage

The vector storage layer is responsible for persisting document embeddings and supporting efficient similarity search operations. To preserve the modular architecture adopted throughout the Storage Manager, this functionality is defined through the abstract class `VectorStore`, which specifies the common interface required by the retrieval pipeline.

The interface exposes the basic operations required by any vector database implementation, including document insertion, similarity search, retrieval of the complete collection, and access to the content hashes used for duplicate detection. As a consequence, replacing the underlying vector

database only requires implementing the same interface, while the higher-level components remain unchanged.

The Proof of Concept adopts ChromaDB as the concrete implementation through the `ChromaStore` class. This wrapper encapsulates all interactions with the database, including collection creation, persistent storage, document insertion, similarity search, and collection management. ChromaDB was selected because it provides persistent storage, native similarity search, efficient metadata management, and seamless integration with Python-based LLM frameworks, making it particularly suitable for rapid prototyping.

During the ingestion phase, each chunk is stored together with its embedding vector and the corresponding metadata. The metadata are preserved inside the vector database and include information such as the document source, page number, chunk identifier, and content hash. This additional information is subsequently exploited during retrieval and traceability operations.

Before new chunks are inserted, the vector store exposes the set of already stored content hashes, allowing the Memory component to efficiently identify duplicate chunks and avoid redundant indexing.

During retrieval, the vector store performs similarity search by comparing the embedding of the user query with the embeddings stored in the collection, returning the most semantically relevant document fragments together with their associated metadata.

Knowledge Memory Management

The Memory component represents the central layer responsible for managing the lifecycle of the knowledge stored inside the Retrieval-Augmented Generation system. Its main purpose is to provide a unified interface for storing, updating, and retrieving information, while hiding the implementation details of the underlying storage and retrieval mechanisms.

Following the modular design adopted throughout the Storage Manager, the memory layer is defined through the abstract class `Memory`. This interface specifies the fundamental operations required by any memory implementation: adding new knowledge items, searching for relevant information, and retrieving the complete stored knowledge base.

This abstraction allows the retrieval pipeline to remain independent from the specific memory implementation. Different types of memories can therefore be introduced in the future, such as domain-specific knowledge bases or agent-specific memories, without requiring changes to the components responsible for document processing or storage.

The concrete implementation developed for the Proof of Concept is represented by the `KBMemory` class, which manages the knowledge base used by the multi-agent system. The class acts as an intermediate layer between the document processing components and the storage mechanisms, coordinating chunk generation, embedding computation, indexing, retrieval, and result refinement.

During the ingestion phase, the memory receives a document path and uses the configured chunker to transform the document into a collection of semantically meaningful chunks. Before inserting new information, the system performs a duplicate detection phase based on the content hash associated with each chunk. This mechanism prevents the same information from being stored multiple times, reducing redundancy and improving the efficiency of the retrieval process.

Only previously unseen chunks are processed further. Their embeddings are generated through

the configured embedding model and stored together with the associated metadata inside the vector database. At the same time, the textual representation of each chunk is inserted into a lexical index based on BM25, enabling the system to combine semantic and keyword-based retrieval strategies.

The retrieval process is designed as a multi-stage pipeline rather than a simple similarity search. The first stage performs semantic retrieval using vector similarity, exploiting the contextual information captured by embeddings. In parallel, a lexical retrieval strategy based on BM25 identifies documents containing relevant keywords from the user query.

For a requested retrieval size of k, both the vector store and the BM25 index independently retrieve $2k$ candidate chunks. The enlarged candidate set increases the probability of recovering relevant documents that may appear only in one of the two rankings before the fusion stage.

The results produced by the two retrieval mechanisms are then combined using the Reciprocal Rank Fusion (RRF) algorithm [13]. This approach allows the system to exploit the complementary strengths of both methods: vector retrieval improves semantic understanding, while lexical retrieval provides accurate matching for specific terms, identifiers, or technical expressions.

Given a document d, its RRF score is computed as

$$RRF(d) = \sum_{r \in R} \frac{1}{k + \text{rank}_r(d)} \tag{4.2}$$

where R denotes the set of ranked retrieval methods, $\text{rank}_r(d)$ is the rank assigned to document d by retrieval method r, and k is a positive constant controlling the influence of lower-ranked documents. In the proposed implementation, the commonly adopted value $k = 60$ is used [13].

Unlike score-based fusion methods, RRF does not require the normalization of similarity scores, which may originate from heterogeneous retrieval models and therefore be expressed on different scales. Instead, the fusion process relies solely on the relative ranking of retrieved documents, making the resulting ranking more robust and less sensitive to differences between lexical and semantic retrieval functions.

The fused ranking is finally refined through a Cross-Encoder reranker: `BAAI/bge-reranker-base`. While the initial retrieval stages evaluate query-document similarity independently, the Cross-Encoder jointly processes the query and each candidate chunk, allowing it to model deeper semantic interactions between the two texts.

This additional ranking stage is computationally more expensive than embedding-based retrieval and is therefore applied only to the limited set of candidate documents produced by the hybrid retrieval phase. The final ranked list returned by the reranker constitutes the contextual information provided to the upper layers of the multi-agent framework.

The resulting memory architecture therefore implements a complete retrieval pipeline composed of:

- semantic retrieval through vector embeddings;
- lexical retrieval through BM25 indexing;
- rank fusion through Reciprocal Rank Fusion;
- relevance refinement through Cross-Encoder reranking.

This multi-stage approach improves the robustness of the knowledge retrieval process, reducing the limitations of relying on a single retrieval strategy and providing higher-quality contextual information to the decision-support agents.

The RAG system is summarized in figure Fig. 4.2

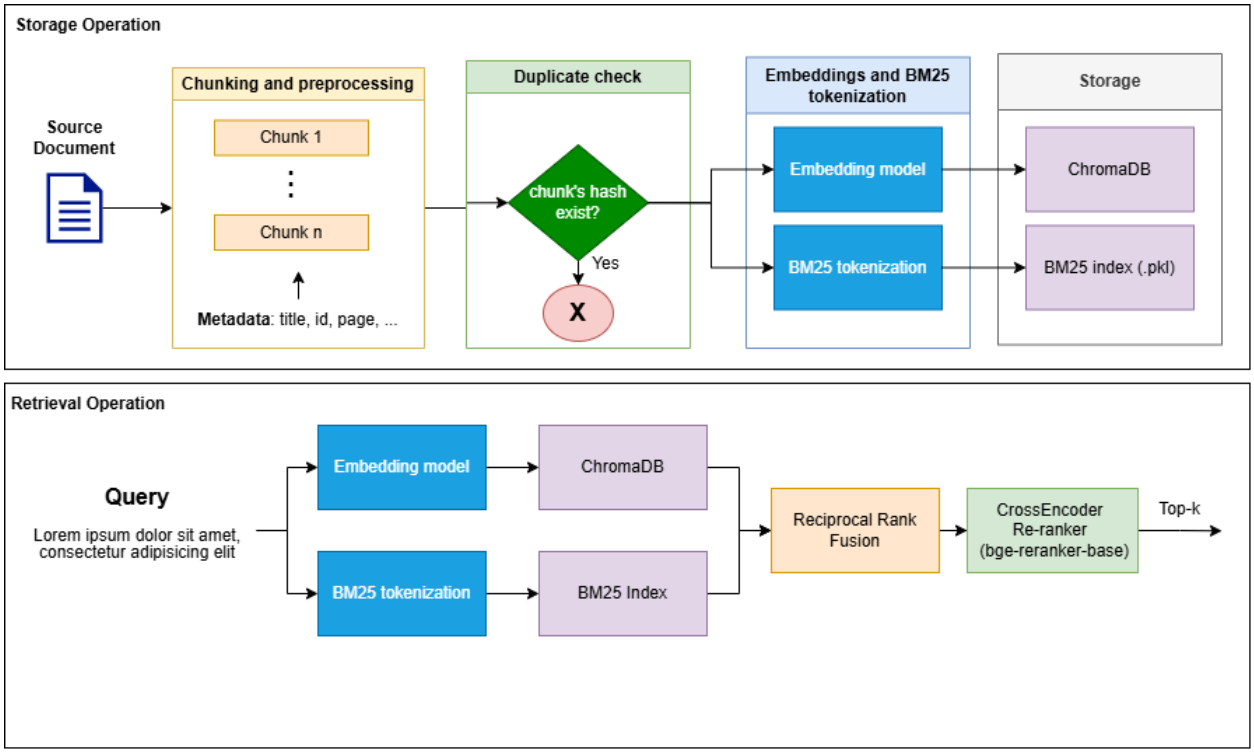

Figure 4.2: Architecture of the Hybrid RAG System. The system integrates a *Storage Pipeline* (top) that cleans, dynamically chunks, and deduplicates raw documents using SHA-1 hashing, indexing unique content simultaneously into a dense vector database (`ChromaDB`) and a sparse lexical in-memory index (`BM25Okapi`, serialized via `pickle`); and a *Retrieval Pipeline* (bottom) that executes parallel dense-semantic and sparse-keyword searches for a given query, merges the candidate results using Reciprocal Rank Fusion (RRF) based on a unified `chunk_id`, and re-ranks them using a Cross-Encoder model (`BAAI/bge-reranker-base`) to output the top-k most contextually relevant passages for the downstream LLM.

RAG Validation

To evaluate the effectiveness of the implemented Retrieval-Augmented Generation pipeline, an experimental validation was conducted on a heterogeneous knowledge base composed of twenty technical documents for a total of approximately 5000 chunks. The dataset included both publicly available ECSS (European Cooperation for Space Standardization) standards and few internal technical documentation provided by Telespazio. The selected corpus reflects the type of documentation that the proposed decision support system is expected to process in real operational scenarios.

A benchmark composed of one hundred queries was manually created for the evaluation. Each query was manually formulated to capture the main semantic content of a randomly selected page while avoiding verbatim copying of the original text. For every query, the expected relevant page was manually annotated and considered as the ground truth.

The objective of the evaluation was to measure the capability of the retrieval pipeline to return

the correct document among the retrieved candidates. Three retrieval configurations were evaluated:

- semantic retrieval using only the vector database;
- hybrid retrieval using Reciprocal Rank Fusion (RRF);
- hybrid retrieval followed by Cross-Encoder reranking.

The performance was assessed using two standard Information Retrieval metrics:

- **Recall@5**, measuring the percentage of queries for which the correct document appeared within the first five retrieved results;
- **Mean Reciprocal Rank (MRR)**, evaluating how early the correct document appeared in the ranking.

Table 4.1: Retrieval performance obtained during the Storage Manager validation.

Configuration	Recall@5 (%)	MRR (%)
Vector Search	79	68.48
Hybrid Search (RRF)	86	71.78
Hybrid Search + Cross Encoder	**86**	**80.50**

Table 4.2: Position of the correct document within the retrieved Top-5 results.

Configuration	Rank1	Rank2	Rank3	Rank4	Rank5
Vector Search	61	11	4	1	2
Hybrid (RRF)	62	14	4	5	1
Hybrid + Reranker	**76**	7	3	0	0

The obtained results highlight the contribution of each stage of the retrieval pipeline.

The introduction of hybrid retrieval through Reciprocal Rank Fusion increased the Recall@5 from 79% to 86%, demonstrating that combining semantic and lexical retrieval allows the system to recover relevant documents that may be overlooked by either retrieval strategy individually.

Although the increase in Mean Reciprocal Rank is more moderate (from 68.48% to 71.78%), the improvement indicates that the correct document is generally positioned closer to the top of the retrieved ranking.

The largest improvement is observed after introducing the Cross-Encoder reranker. While the Recall@5 remains unchanged, the MRR increases significantly from 71.78% to 80.50%. This behaviour is expected since the reranker does not retrieve additional documents but rather reorders the candidates already returned by the hybrid retrieval stage.

The position distribution further confirms this behaviour. The number of queries for which the correct document is ranked first increases from 62 to 76, while no relevant document appears in the fourth or fifth position after reranking. This demonstrates that the Cross-Encoder is particularly effective at improving the ranking quality of already retrieved candidates.

Limitations

Although the proposed pipeline achieves encouraging retrieval performance, the evaluation has been conducted on a relatively limited corpus composed of twenty technical documents (around 5000 chunks). Future work should assess scalability on substantially larger document collections and investigate the impact of alternative embedding models and reranking strategies.

4.2.3 Additional Data Interfaces

Although the Retrieval-Augmented Generation pipeline represents the main knowledge management component of the Storage Manager, the module is not limited to document-based information retrieval. The decision-support system requires access to heterogeneous data sources, including operational data, planning information, and system status information.

Since for the PoC capabilities is not required to ingest data from the real data sources of Telespazio, we decided to use standard local files for the data sources to be ingested in the system. The Storage Manager was designed as a unified data access layer, exposing high-level interfaces for retrieving different types of information while hiding the specific format and parsing logic of each source.

The implemented Proof of Concept supports the ingestion and retrieval of structured and semi-structured data, including:

- planning informations stored in CSV files;
- network data config of the ground segment
- open tickets of the ground segment

Each data source is handled through dedicated processing functions responsible for parsing the original format and transforming the information into a representation suitable for consumption by the upper layers of the architecture.

For example, planning data can be filtered according to operational parameters such as time interval, satellite identifier, or ground station, allowing agents to retrieve only the information relevant to the current decision-support task.

Unlike the knowledge base, these data sources are not indexed through semantic retrieval mechanisms, since they represent structured operational information where explicit filtering and deterministic retrieval provide a more appropriate access strategy.

The Storage Manager therefore acts as an abstraction layer over both unstructured knowledge sources and structured operational data, providing a single entry point for the AI components. This design prevents agents from directly interacting with heterogeneous data formats, improving maintainability and allowing additional data sources to be integrated without modifying the reasoning layer.

4.2.4 Chat Storage

Besides managing external knowledge and operational data, the Storage Manager provides a persistent storage layer for the conversational information generated during the interaction between users and the multi-agent system.

The Chat Storage component has been designed to maintain the history of interactions, enabling users to retrieve previous conversations and allowing the AI components to access contextual information related to ongoing tasks. Unlike the Knowledge Base memory, which contains stable domain information retrieved through the RAG pipeline, chat storage manages dynamic information generated during system execution.

The implemented solution relies on `MongoDB`, a document-oriented NoSQL database selected due to its flexibility in managing conversational data, where messages may contain heterogeneous metadata depending on the involved agent, language model, or execution context.

The architecture separates conversation-level information from individual message data through two different collections:

- `chats`, storing high-level information related to each conversation;
- `messages`, storing the individual messages exchanged during the interaction.

Each conversation is uniquely identified by a `chat_id` and contains information such as the associated user, the agent involved in the interaction, creation and update timestamps, and aggregate statistics including the number of exchanged messages and the total number of processed tokens.

Individual messages are stored as independent documents containing the conversation identifier, user identifier, message role, textual content, and additional execution metadata. The stored information includes the language model used to generate the response, the number of input and output tokens, the response latency, and optional metadata generated during the execution.

The inclusion of execution-related information enables the system to monitor the interaction history and provides the basis for future extensions such as performance analysis, cost estimation, and agent behaviour evaluation.

To improve retrieval efficiency, the database defines dedicated indexes on the main query fields. In particular, indexes are created on the conversation identifier and on the combination of conversation identifier and creation timestamp, allowing chronological reconstruction of conversations with limited query overhead. Additionally, user-based indexing enables efficient retrieval of all conversations associated with a specific user.

The main operations exposed by the Chat Storage component are:

- creation of new conversations associated with a user and a selected agent;
- persistent storage of user and assistant messages;
- chronological retrieval of messages belonging to a conversation;
- retrieval of the conversation history associated with a user.

The separation between conversational storage and knowledge retrieval follows the principle of information specialization. While the RAG memory is optimized for semantic search over external documents, the Chat Storage component is optimized for temporal reconstruction of previous interactions.

This distinction is particularly important in a multi-agent decision-support system, where conversational context, user requests, and generated responses represent operational information rather

than static domain knowledge. The proposed design allows both components to evolve independently, enabling future extensions such as conversation summarization, user-specific preferences, or persistent agent memory mechanisms.

4.2.5 Storage Manager Limitations

Despite the modular architecture adopted for the Storage Manager, the current implementation presents some limitations related to the scope of the Proof of Concept and the characteristics of the available data sources.

First, the current document ingestion pipeline mainly focuses on PDF, DOCX, and TXT formats. Although the chunking architecture allows the integration of additional loaders, more complex formats such as HTML pages, engineering databases, or domain-specific document structures are not yet directly supported.

Second, the implemented Retrieval-Augmented Generation pipeline relies on a general-purpose embedding model, namely *all-MiniLM-L6-v2*. While this model provides a suitable balance between computational efficiency and semantic retrieval capability, it is not specifically optimized for aerospace engineering terminology. Domain-specific embedding models could potentially improve retrieval performance on highly technical documentation.

Furthermore, the current validation campaign was performed on a limited knowledge base composed of twenty technical documents and one hundred manually generated queries. Although the obtained results demonstrate the effectiveness of the proposed retrieval strategy, a larger benchmark containing a higher variety of documents and automatically generated evaluation scenarios would provide a more comprehensive assessment.

Finally, the current Storage Manager implementation focuses primarily on retrieval and data accessibility. Advanced knowledge management capabilities, such as automatic document updating, conflict detection between different sources, and temporal versioning of knowledge, represent possible future extensions required for deployment in a fully operational environment.

These limitations do not affect the objectives of the Proof of Concept, whose main goal is to demonstrate the feasibility of a modular decision-support architecture. However, they highlight the main aspects requiring further development before industrial adoption.

4.3 AI Manager Module

Bibliography

[1] A. Alansari and H. Luqman. Large language models hallucination: A comprehensive survey. *arXiv preprint arXiv:2510.06265*, 2025.

[2] M. Alkayyal, S. Malberg, and G. Groh. An llm-based decision support system for strategic decision-making. In *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*, pages 460–464. Springer, 2025.

[3] D. Anh-Hoang, V. Tran, and L.-M. Nguyen. Survey and analysis of hallucinations in large language models: attribution to prompting strategies or model behavior. *Frontiers in Artificial Intelligence*, Volume 8 - 2025, 2025.

[4] Y. Bengio, R. Ducharme, P. Vincent, and C. Jauvin. A neural probabilistic language model. *Journal of machine learning research*, 3(Feb):1137–1155, 2003.

[5] M. Besta, N. Blach, A. Kubicek, R. Gerstenberger, M. Podstawski, L. Gianinazzi, J. Gajda, T. Lehmann, H. Niewiadomski, P. Nyczyk, et al. Graph of thoughts: Solving elaborate problems with large language models. In *Proceedings of the AAAI conference on artificial intelligence*, volume 38, pages 17682–17690, 2024.

[6] O. Bianchi, M. J. Koretsky, M. Willey, C. X. Alvarado, T. Nayak, A. Asija, N. Kuznetsov, M. A. Nalls, F. Faghri, and D. Khashabi. Hidden in the haystack: Smaller needles are more difficult for llms to find. *arXiv preprint arXiv:2505.18148*, 2025.

[7] D. A. Boiko, R. MacKnight, B. Kline, and G. Gomes. Autonomous chemical research with large language models. *Nature*, 624(7992):570–578, 2023.

[8] P. Bojanowski, E. Grave, A. Joulin, and T. Mikolov. Enriching word vectors with subword information. *CoRR*, abs/1607.04606, 2016.

[9] R. Bommasani, D. A. Hudson, E. Adeli, R. Altman, S. Arora, S. von Arx, M. S. Bernstein, J. Bohg, A. Bosselut, E. Brunskill, et al. On the opportunities and risks of foundation models. *arXiv preprint arXiv:2108.07258*, 2021.

[10] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.

[11] S. Chen, S. Wong, L. Chen, and Y. Tian. Extending context window of large language models via positional interpolation, 2023.

[12] P. F. Christiano, J. Leike, T. Brown, M. Martic, S. Legg, and D. Amodei. Deep reinforcement learning from human preferences. *Advances in neural information processing systems*, 30, 2017.

[13] G. V. Cormack, C. L. Clarke, and S. Buettcher. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of the 32nd international ACM SIGIR conference on Research and development in information retrieval*, pages 758–759, 2009.

[14] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, editors, *Advances in Neural Information Processing Systems*, volume 35, pages 16344–16359. Curran Associates, Inc., 2022.

[15] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, pages 4171–4186, 2019.

[16] E. Eigner and T. Händler. Determinants of llm-assisted decision-making. *arXiv preprint arXiv:2402.17385*, 2024.

[17] J. L. Elman. Finding structure in time. *Cognitive science*, 14(2):179–211, 1990.

[18] European Cooperation for Space Standardization. Space Engineering: System Engineering General Requirements. ECSS Standard ECSS-E-ST-10C Rev.1, ESA Requirements and Standards Division, ESTEC, Noordwijk, The Netherlands, Feb. 2017.

[19] European Cooperation for Space Standardization. Space Engineering: Software. ECSS Standard ECSS-E-ST-40C Rev.1, ESA Requirements and Standards Division, ESTEC, Noordwijk, The Netherlands, Apr. 2025.

[20] European Cooperation for Space Standardization. Space Product Assurance: Software Product Assurance. ECSS Standard ECSS-Q-ST-80C Rev.2, ESA Requirements and Standards Division, ESTEC, Noordwijk, The Netherlands, Apr. 2025.

[21] G. A. Fink. n-gram models. In *Markov Models for Pattern Recognition: From Theory to Applications*, pages 107–127. Springer, 2014.

[22] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh. Gptq: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323*, 2022.

[23] M. Gordon, K. Duh, and N. Andrews. Compressing bert: Studying the effects of weight pruning on transfer learning. In *Proceedings of the 5th Workshop on Representation Learning for NLP*, pages 143–155, 2020.

[24] R. Grosse. Lecture 15: Exploding and vanishing gradients. *University of Toronto Computer Science*, 2017.

[25] Z. Han, C. Gao, J. Liu, J. Zhang, and S. Q. Zhang. Parameter-efficient fine-tuning for large models: A comprehensive survey. *arXiv preprint arXiv:2403.14608*, 2024.

[26] Z. S. Harris. Distributional structure. *Word*, 10(2-3):146–162, 1954.

[27] G. Hinton, O. Vinyals, and J. Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.

[28] S. Hochreiter and J. Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.

[29] J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai, E. Rutherford, D. Casas, L. A. Hendricks, J. Welbl, A. Clark, et al. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 10, 2022.

[30] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen, et al. Lora: Low-rank adaptation of large language models. *Iclr*, 1(2):3, 2022.

[31] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung. Survey of hallucination in natural language generation. *ACM computing surveys*, 55(12):1–38, 2023.

[32] Z. Jiang, J. Araki, H. Ding, and G. Neubig. How can we know when language models know? on the calibration of language models for question answering. *Transactions of the Association for Computational Linguistics*, 9:962–977, 09 2021.

[33] A. Kalyuzhnaya, S. Mityagin, E. Lutsenko, A. Getmanov, Y. Aksenkin, K. Fatkhiev, K. Fedorin, N. O. Nikitin, N. Chichkova, V. Vorona, and A. Boukhanovsky. Llm agents for smart city management: Enhancing decision support through multi-agent ai systems. *Smart Cities*, 8(1), 2025.

[34] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.

[35] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474, 2020.

[36] J. Li, J. Xu, S. Huang, Y. Chen, W. Li, J. Liu, Y. Lian, J. Pan, L. Ding, H. Zhou, et al. Large language model inference acceleration: A comprehensive hardware perspective. *arXiv preprint arXiv:2410.04466*, 2024.

[37] K. Li and R. S. Whittle. Toward an Earth-Independent System for EVA Mission Planning: Integrating Physical Models, Domain Knowledge, and Agentic RAG to Provide Explainable LLM-Based Decision Support. In L. Bensch, T. Nilsson, M. Nisser, P. Pataranutaporn, A. Schmidt, and V. Sumini, editors, *Advancing Human-Computer Interaction for Space Exploration (SpaceCHI 2025)*, volume 130 of *Open Access Series in Informatics (OASIcs)*, pages 6:1–6:17, Dagstuhl, Germany, 2025. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.

[38] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han. Awq: Activation-aware weight quantization for on-device llm compression and

acceleration. In P. Gibbons, G. Pekhimenko, and C. D. Sa, editors, *Proceedings of Machine Learning and Systems*, volume 6, pages 87–100, 2024.

[39] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.

[40] C. D. Manning. *Introduction to information retrieval*. Syngress Publishing,, 2008.

[41] D. Maranto. Llmsat: A large language model-based goal-oriented agent for autonomous space exploration. *arXiv preprint arXiv:2405.01392*, 2024.

[42] T. Mikolov, K. Chen, G. Corrado, and J. Dean. Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*, 2013.

[43] Ç. U. Öğdü, K. Arslanoğlu, and M. Karaköse. An adaptive multi-agent llm-based clinical decision support system integrating biomedical rag and web intelligence. *IEEE Access*, 2025.

[44] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744, 2022.

[45] J. S. Park, J. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, UIST '23, New York, NY, USA, 2023. Association for Computing Machinery.

[46] B. Peng, J. Quesnelle, H. Fan, and E. Shippole. Yarn: Efficient context window extension of large language models, 2023.

[47] J. Pennington, R. Socher, and C. D. Manning. Glove: Global vectors for word representation. In *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*, pages 1532–1543, 2014.

[48] M. E. Peters, M. Neumann, M. Iyyer, M. Gardner, C. Clark, K. Lee, and L. Zettlemoyer. Deep contextualized word representations. In M. Walker, H. Ji, and A. Stent, editors, *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, pages 2227–2237, New Orleans, Louisiana, June 2018. Association for Computational Linguistics.

[49] W. A. Qader, M. M. Ameen, and B. I. Ahmed. An overview of bag of words; importance, implementation, applications, and challenges. In *2019 international engineering conference (IEC)*, pages 200–204. IEEE, 2019.

[50] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.

[51] R. Rafailov, A. Sharma, E. Mitchell, C. D. Manning, S. Ermon, and C. Finn. Direct preference optimization: Your language model is secretly a reward model. *Advances in neural information processing systems*, 36:53728–53741, 2023.

[52] D. E. Rumelhart, G. E. Hinton, and R. J. Williams. Learning representations by back-propagating errors. *nature*, 323(6088):533–536, 1986.

[53] V. Sanh, L. Debut, J. Chaumond, and T. Wolf. Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*, 2019.

[54] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in neural information processing systems*, 36:68539–68551, 2023.

[55] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[56] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems*, 36:8634–8652, 2023.

[57] S. T. Sreenivas, S. Muralidharan, R. Joshi, M. Chochowski, A. S. Mahabaleshwarkar, G. Shen, J. Zeng, Z. Chen, Y. Suhara, S. Diao, et al. Llm pruning and distillation in practice: The minitron approach. *arXiv preprint arXiv:2408.11796*, 2024.

[58] N. Stiennon, L. Ouyang, J. Wu, D. Ziegler, R. Lowe, C. Voss, A. Radford, D. Amodei, and P. F. Christiano. Learning to summarize with human feedback. *Advances in neural information processing systems*, 33:3008–3021, 2020.

[59] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.

[60] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[61] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, et al. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6):186345, 2024.

[62] S. Wang, B. Z. Li, M. Khabsa, H. Fang, and H. Ma. Linformer: Self-attention with linear complexity. *arXiv preprint arXiv:2006.04768*, 2020.

[63] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*, 2022.

[64] J. Wei, Y. Tay, R. Bommasani, C. Raffel, B. Zoph, S. Borgeaud, D. Yogatama, M. Bosma, D. Zhou, D. Metzler, et al. Emergent abilities of large language models. *arXiv preprint arXiv:2206.07682*, 2022.

[65] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.

[66] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han. Smoothquant: Accurate and efficient post-training quantization for large language models. In *International conference on machine learning*, pages 38087–38099. PMLR, 2023.

[67] Y. Xu, L. Xie, X. Gu, X. Chen, H. Chang, H. Zhang, Z. Chen, X. Zhang, and Q. Tian. Qa-lora: Quantization-aware low-rank adaptation of large language models. *arXiv preprint arXiv:2309.14717*, 2023.

[68] S. Yao, D. Yu, J. Zhao, I. Shafran, T. Griffiths, Y. Cao, and K. Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. *Advances in neural information processing systems*, 36:11809–11822, 2023.

[69] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. R. Narasimhan, and Y. Cao. React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations*, 2022.